Datos Masivos

Valdés Uriarte, Santiago 0212570	Isunza Frank, Carlos 0212508

Proyecto Final:

Potencial en los Jugadores Jóvenes

UNIVERSIDAD Panamericana

EXPLORATORY DATA ANALYSIS 1

TROUBLESHOOTING 32

EXPLORATORY DATA ANALYSIS

EDA TABLA ESTÁTICA

Python
```python
from pyspark.sql import SparkSession

# Configura la sesión de Spark
spark = SparkSession.builder \
    .appName("Análisis de FIFA Players con SQL") \
    .getOrCreate()

# Cargar datos desde HDFS o local, ajusta la ruta según donde tengas el
archivo
df = spark.read.format("csv") \
    .option("header", "true") \
    .load("hdfs:///user/usuario/datasets/csv/male_players.csv")

# Crear una vista temporal de SQL
df.createOrReplaceTempView("fifa_players")
```

Python
```python
# Consulta para encontrar el top n% de jugadores (por ejemplo, top 5%)
percentile = 95  # Ejemplo: Top 5%
query_top_players = f"""
    SELECT *
    FROM fifa_players
    WHERE overall >= (
        SELECT approx_percentile(overall, {percentile}/100.0)
        FROM fifa_players
    )
"""
top_players_df = spark.sql(query_top_players)

# Atributos relevantes para el análisis
relevant_attributes = [
    "movement_agility", "skill_ball_control", "power_strength",
    "attacking_crossing", "attacking_finishing",
"attacking_heading_accuracy",
    "defending_marking_awareness", "defending_standing_tackle",
"defending_sliding_tackle",
    "mentality_aggression", "mentality_interceptions",
"mentality_positioning"
```

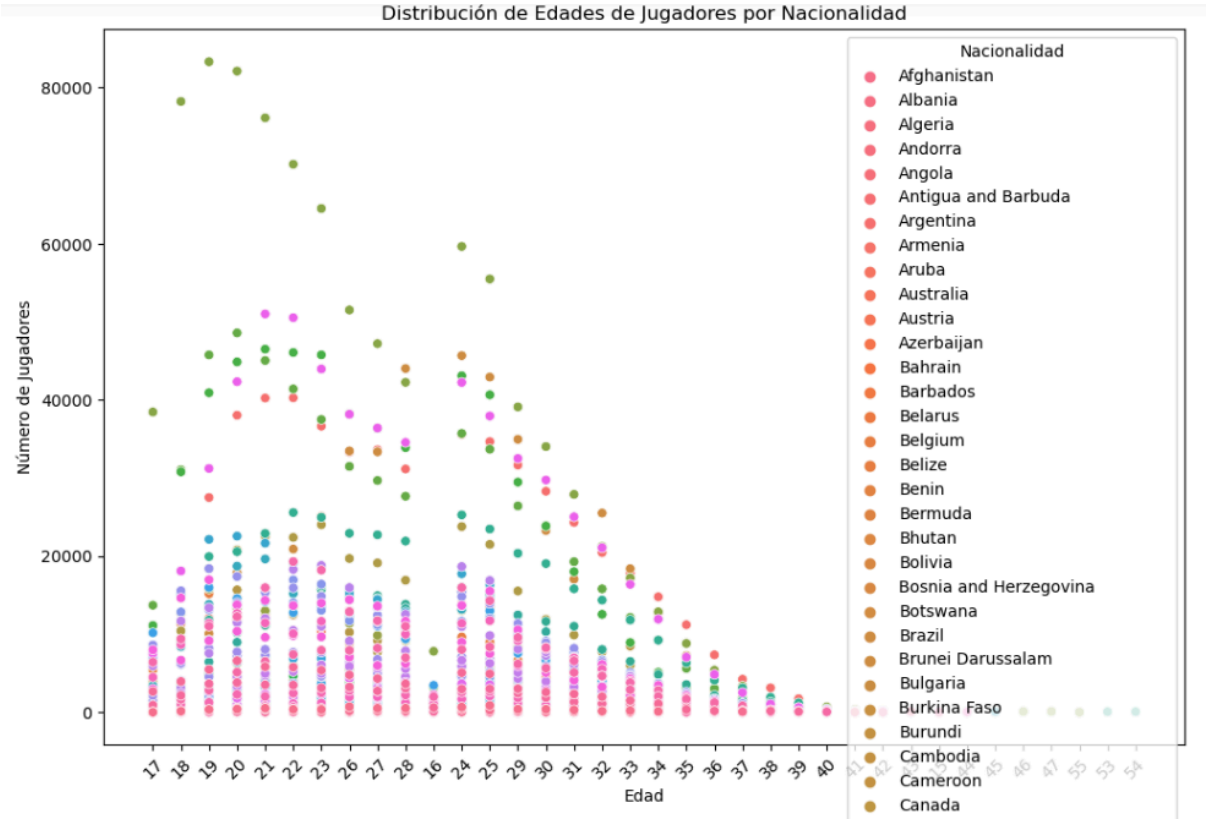
```
]

# Consulta para calcular estadísticas resumidas de los atributos relevantes
queries_summary_stats = []
for attribute in relevant_attributes:
    query_summary_stats = f"""
        SELECT
            '{attribute}' AS attribute,
            AVG({attribute}) AS avg_value,
            MIN({attribute}) AS min_value,
            MAX({attribute}) AS max_value,
            STDDEV({attribute}) AS stddev_value
        FROM fifa_players
        WHERE player_id IN (
            SELECT player_id
            FROM ({query_top_players}) top_players
        )
    """
    queries_summary_stats.append(query_summary_stats)

# Ejecutar las consultas y almacenar los resultados en un DataFrame de
pandas
import pandas as pd

summary_stats = pd.DataFrame()
for query in queries_summary_stats:
    df_stats = spark.sql(query).toPandas()
    summary_stats = pd.concat([summary_stats, df_stats])

# Mostrar las estadísticas resumidas
print(summary_stats)
```

	attribute	avg_value	min_value	max_value	stddev_value
0	movement_agility	68.062864	18	96	14.402480
0	skill_ball_control	68.415261	10	96	17.677782
0	power_strength	69.847878	20	96	11.842788
0	attacking_crossing	58.869045	10	94	20.014196
0	attacking_finishing	54.554628	10	95	21.325068
0	attacking_heading_accuracy	59.405995	10	95	19.384359
0	defending_marking_awareness	51.583096	10	95	23.549605
0	defending_standing_tackle	53.920070	10	94	24.273320
0	defending_sliding_tackle	51.063540	10	95	24.394017
0	mentality_aggression	64.087870	11	96	17.863160
0	mentality_interceptions	54.242018	10	93	22.850917
0	mentality_positioning	59.332988	10	96	21.605165

```python
# Consulta SQL para obtener la distribución de edades por nacionalidad
query_age_distribution = """
    SELECT nationality_name, age, COUNT(*) AS num_players
    FROM fifa_players
    GROUP BY nationality_name, age
    ORDER BY nationality_name, age
"""

# Ejecutar la consulta y obtener los resultados
age_distribution_df = spark.sql(query_age_distribution).toPandas()

# Mostrar los primeros registros para verificar
print(age_distribution_df.head())
```

```
  nationality_name age  num_players
0      Afghanistan  17           31
1      Afghanistan  18           57
2      Afghanistan  19          142
3      Afghanistan  20          131
4      Afghanistan  21          122
```

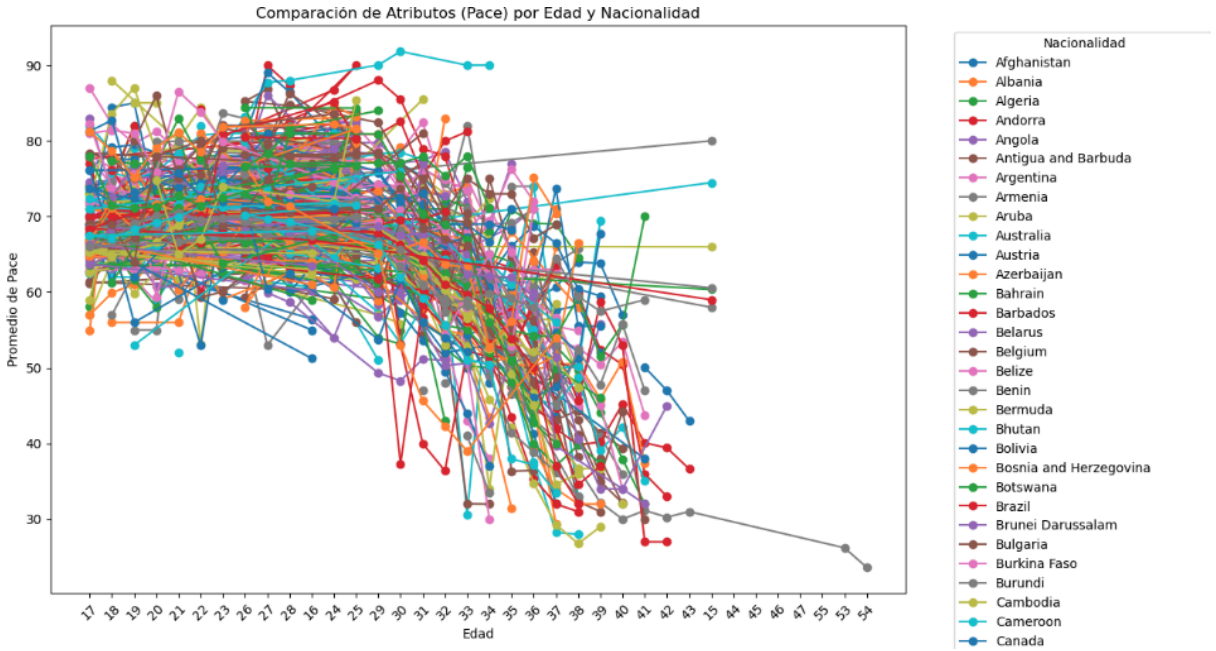
```python
import matplotlib.pyplot as plt
import seaborn as sns

# Crear un gráfico de dispersión para mostrar la distribución de edades por nacionalidad
plt.figure(figsize=(12, 8))
sns.scatterplot(x='age', y='num_players', hue='nationality_name', data=age_distribution_df)
plt.title('Distribución de Edades de Jugadores por Nacionalidad')
plt.xlabel('Edad')
plt.ylabel('Número de Jugadores')
plt.xticks(rotation=45)
plt.legend(title='Nacionalidad')
plt.tight_layout()
plt.show()
```



```python
# Consulta SQL para obtener los atributos promedio por edad y nacionalidad
query_avg_attributes = """
    SELECT nationality_name, age,
           AVG(pace) AS avg_pace,
           AVG(shooting) AS avg_shooting,
           AVG(passing) AS avg_passing,
           AVG(dribbling) AS avg_dribbling
    FROM fifa_players
    GROUP BY nationality_name, age
    ORDER BY nationality_name, age
"""

# Ejecutar la consulta y obtener los resultados
avg_attributes_df = spark.sql(query_avg_attributes).toPandas()

# Mostrar los primeros registros para verificar
print(avg_attributes_df.head())
```

```
  nationality_name age   avg_pace  avg_shooting  avg_passing  avg_dribbling
0      Afghanistan  17  69.000000     51.000000    43.000000      55.000000
1      Afghanistan  18  72.508772     51.982456    44.000000      59.964912
2      Afghanistan  19  72.140845     49.260563    53.704225      59.971831
3      Afghanistan  20  70.709924     53.030534    54.671756      59.389313
4      Afghanistan  21  70.918033     56.065574    58.049180      62.934426
```

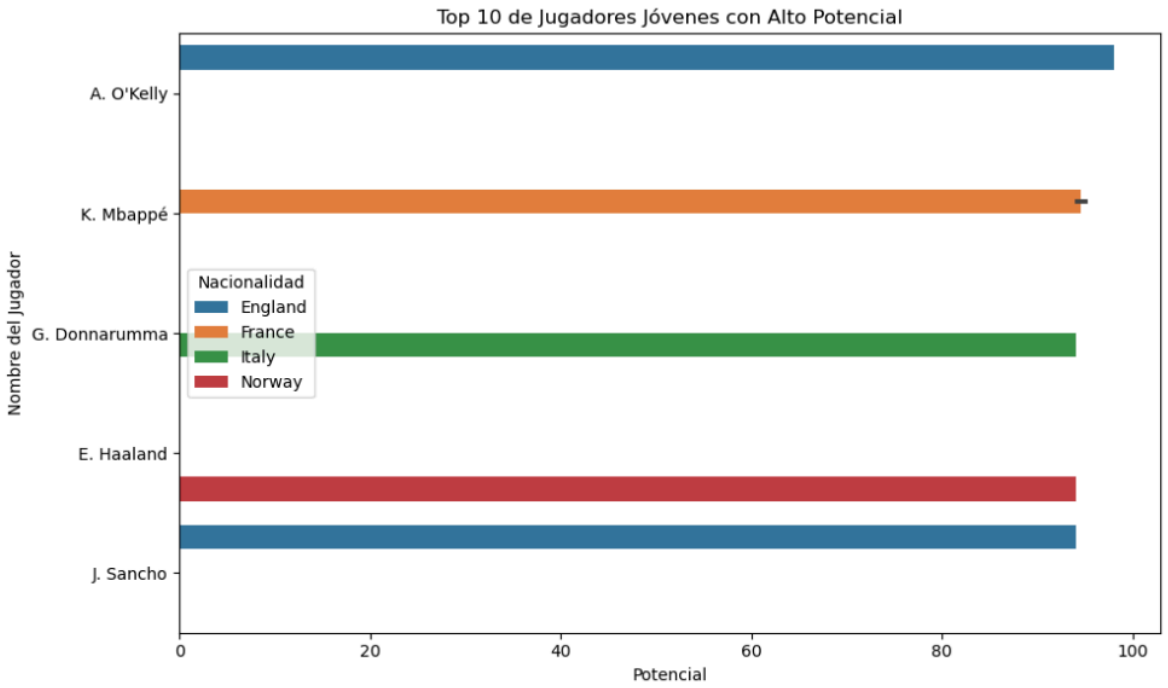
```python
# Crear gráficos de línea para comparar atributos por edad y nacionalidad
plt.figure(figsize=(12, 8))
for nationality, data in avg_attributes_df.groupby('nationality_name'):
    plt.plot(data['age'], data['avg_pace'], marker='o', label=nationality)

plt.title('Comparación de Atributos (Pace) por Edad y Nacionalidad')
plt.xlabel('Edad')
plt.ylabel('Promedio de Pace')
plt.xticks(rotation=45)
plt.legend(title='Nacionalidad', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



```python
# Consulta SQL para encontrar jugadores menores de 21 años con alto potencial
query_young_players = """
    SELECT short_name, nationality_name, age, potential
    FROM fifa_players
    WHERE age < 21
    GROUP BY short_name, age, nationality_name, potential
    ORDER BY potential DESC
    LIMIT 10
"""

# Ejecutar la consulta y obtener los resultados
young_players_df = spark.sql(query_young_players).toPandas()

# Mostrar los primeros registros para verificar
print(young_players_df.head())
```

```
      short_name nationality_name age potential
0     A. O'Kelly          England  16        98
1      K. Mbappé           France  18        95
2      K. Mbappé           France  19        95
3      K. Mbappé           France  20        95
4  G. Donnarumma            Italy  19        94
```

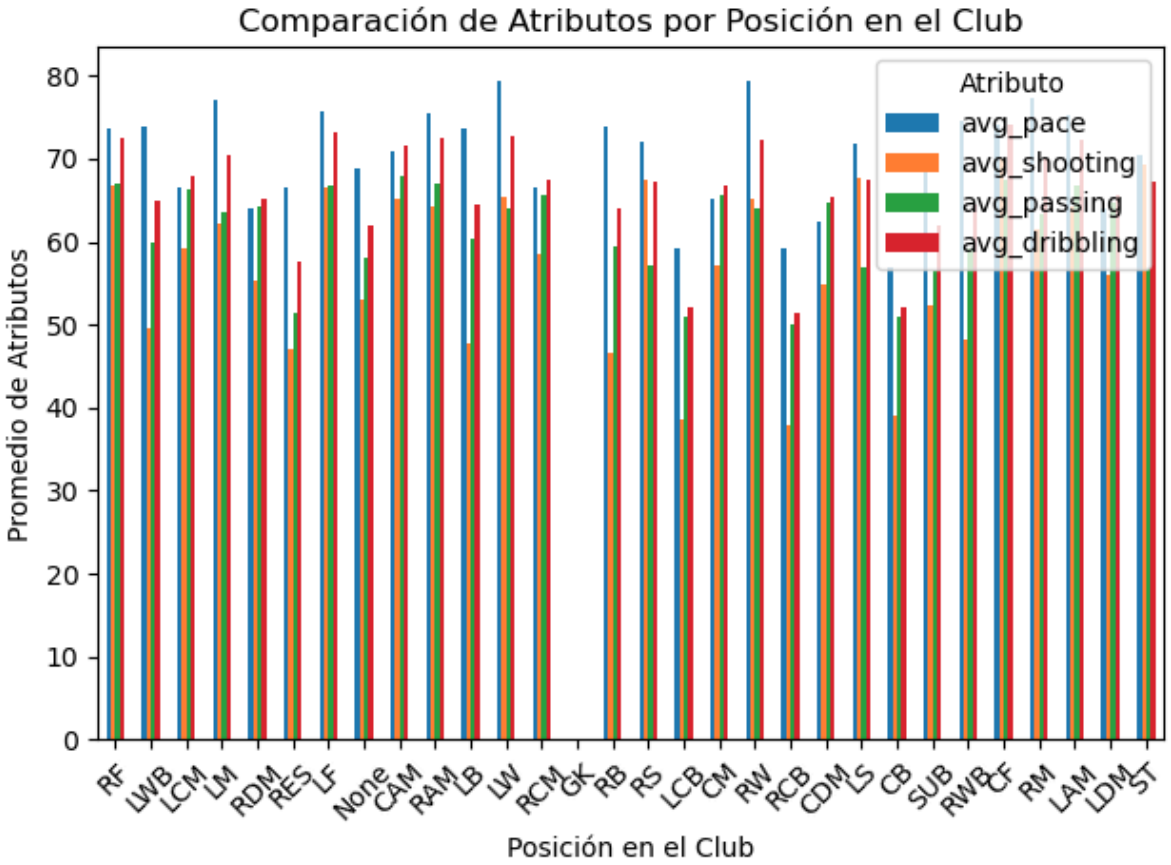
```python
# Crear un gráfico de barras horizontales para mostrar los jugadores jóvenes con alto potencial
plt.figure(figsize=(10, 6))
sns.barplot(x='potential', y='short_name', hue='nationality_name', data=young_players_df)
plt.title('Top 10 de Jugadores Jóvenes con Alto Potencial')
plt.xlabel('Potencial')
plt.ylabel('Nombre del Jugador')
plt.legend(title='Nacionalidad')
plt.tight_layout()
plt.show()
```



```python
# Consulta SQL para comparar atributos promedio por posición en el club
query_position_attributes = """
    SELECT club_position,
           AVG(pace) AS avg_pace,
           AVG(shooting) AS avg_shooting,
           AVG(passing) AS avg_passing,
           AVG(dribbling) AS avg_dribbling
    FROM fifa_players
    GROUP BY club_position
"""

# Ejecutar la consulta y obtener los resultados
position_attributes_df = spark.sql(query_position_attributes).toPandas()

# Mostrar los primeros registros para verificar
print(position_attributes_df.head())
```

	club_position	avg_pace	avg_shooting	avg_passing	avg_dribbling
0	RF	73.644584	66.884607	67.093957	72.531243
1	LWB	73.987064	49.509206	60.003380	64.862458
2	LCM	66.590727	59.215875	66.225143	67.915363
3	LM	77.146174	62.116753	63.621365	70.352625
4	RDM	64.052628	55.365526	64.275610	65.225139

```python
# Crear un gráfico de barras apiladas para comparar atributos por posición en el club
plt.figure(figsize=(12, 8))
position_attributes_df.set_index('club_position')[['avg_pace', 'avg_shooting', 'avg_passing', 'avg_dribbling']].plot(kind='bar', stacked=False)
plt.title('Comparación de Atributos por Posición en el Club')
plt.xlabel('Posición en el Club')
plt.ylabel('Promedio de Atributos')
plt.xticks(rotation=45)
plt.legend(title='Atributo')
plt.tight_layout()
plt.show()
```

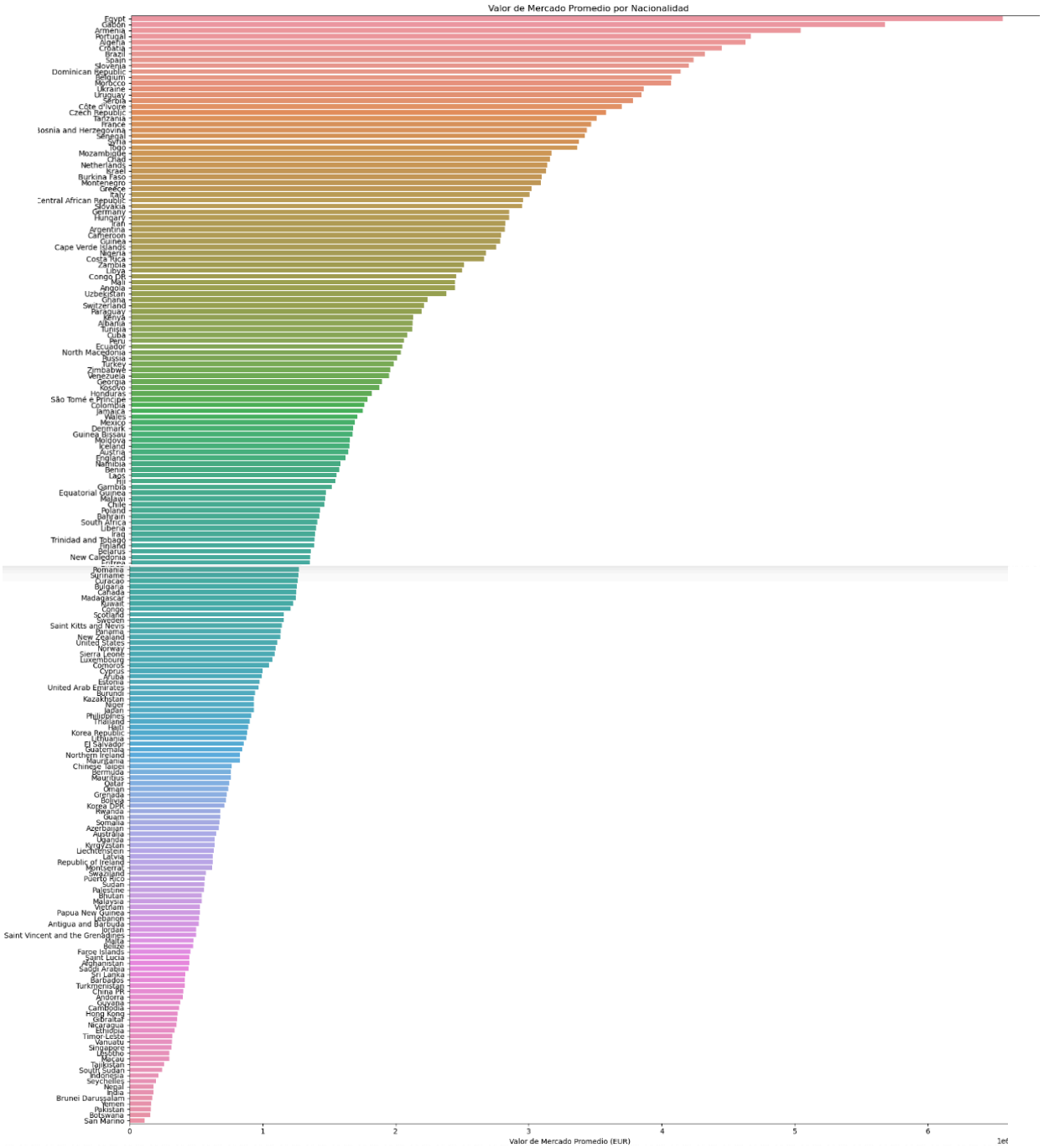


```python
# Consulta SQL para obtener el valor de mercado promedio por nacionalidad
query_avg_value_by_nationality = """
    SELECT nationality_name,
           AVG(value_eur) AS avg_value_eur
    FROM fifa_players
    GROUP BY nationality_name
    ORDER BY avg_value_eur DESC
"""

# Ejecutar la consulta y obtener los resultados
avg_value_by_nationality_df =
spark.sql(query_avg_value_by_nationality).toPandas()

# Mostrar los primeros registros para verificar
print(avg_value_by_nationality_df.head())
```

```
  nationality_name  avg_value_eur
0            Egypt   6.304749e+06
1            Gabon   5.451815e+06
2          Armenia   4.843683e+06
3         Portugal   4.483910e+06
4          Algeria   4.444989e+06
```

Python

```python
# Crear un gráfico de barras horizontales para mostrar el valor de mercado promedio por nacionalidad
plt.figure(figsize=(20, 22))
sns.barplot(x='avg_value_eur', y='nationality_name', data=avg_value_by_nationality_df)
plt.title('Valor de Mercado Promedio por Nacionalidad')
plt.xlabel('Valor de Mercado Promedio (EUR)')
plt.ylabel('Nacionalidad')
plt.tight_layout()
plt.show()
```



CONCLUSIÓN

Distribución de Edades por Nacionalidad

1. *Nacionalidades con Mayor Cantidad de Jugadores Jóvenes*:

   Inglaterra, España, Alemania y Brasil son las naciones con mayor cantidad de jugadores jóvenes menores de 23 años. Esto refleja la solidez de sus sistemas de desarrollo juvenil y la cantidad de talento que producen consistentemente.

2. *Patrones Notables en la Distribución de Edades:*

   La distribución de edades por nacionalidad muestra que la mayoría de las naciones tienen su pico de jugadores entre los 20 y 21 años, después del cual la cantidad de jugadores disminuye gradualmente. Este patrón sugiere que estas edades son cruciales para la transición de jugadores jóvenes a profesionales establecidos.

Promedios de Atributos por Edad y Nacionalidad

1. *Valores Promedio de Atributos por Edad:*

   Los valores promedio de pace y otros atributos comienzan alrededor de 70 a los 17 años y tienden a subir hasta los 26 años. En raras ocasiones, estos valores continúan subiendo o se mantienen entre los 26 y 29 años, pero generalmente empiezan a disminuir a partir de los 30 años. Este patrón refleja el desarrollo físico y técnico de los jugadores jóvenes y el impacto del envejecimiento en su rendimiento.

2. *Nacionalidades que Destacan en Atributos Específicos:*

   Brasil, Camerún, Camboya, Ghana y Portugal destacan en el atributo de pace. Francia, aunque no destaca específicamente en pace, es notable por mantener un

nivel constante en este atributo a lo largo de los años. Esto sugiere una formación continua y efectiva en estos países.

Top 10 Jugadores Jóvenes con Alto Potencial

1. *Lista de Jugadores con Mayor Potencial:*

   Jugadores como Kylian Mbappé (Francia), Gianluigi Donnarumma (Italia), Erling Haaland (Noruega) y Jadon Sancho (Inglaterra) se destacan repetidamente a lo largo de diferentes ediciones del juego, lo que subraya la importancia de la consistencia en el potencial.

   Los atributos destacados de estos jugadores incluyen pace, dribbling, agilidad y remates para Mbappé; fuerza, cabeceo, remates y tiro largo para Haaland; pace, dribbling y agilidad para Sancho; y todas las métricas de portero para Donnarumma, quien también destaca por su altura de 1.96m.

Promedios de Atributos por Posición

1. *Valores Promedio de Atributos por Posición:*

   Atributos promedio para diferentes posiciones muestran variaciones significativas. Por ejemplo, las posiciones de banda (LWB, LM, LF, RAM, LW, RW, RM) tienen altos valores de pace y dribbling, reflejando la necesidad de velocidad y habilidad para el regate en estos roles.

   Otros roles, como los delanteros (ST, CF) y centrocampistas (CM, CAM), tienen una mezcla equilibrada de atributos técnicos y físicos.

Valor Promedio por Nacionalidad

1. *Nacionalidades con Valores Promedio Más Altos y Más Bajos:*

   Las nacionalidades con valores promedio más altos incluyen Egipto, Gabón, Armenia, Portugal, Argelia, Croacia, Brasil, España, Eslovenia y Bélgica. Estas naciones tienden a tener uno o dos jugadores estrella que influyen significativamente en el valor promedio.

   Las nacionalidades con valores promedio más bajos incluyen San Marino, Botswana, Pakistán, Yemen y Brunei. Sin embargo, estos valores deben ser interpretados con precaución debido a la presencia de sesgos por jugadores atípicos.

2. *Sesgo Notable en los Datos:*

   Ejemplos de sesgos incluyen a Egipto con Salah, Gabón con Aubameyang, Armenia con Mkhitaryan, Argelia con Mahrez, y Eslovenia con Oblak y Handanovic. La presencia de estos jugadores estrella puede distorsionar la percepción del valor promedio de sus respectivos países.

Correlaciones Específicas

1. *Correlaciones entre Atributos y Potencial:*

   Hay una fuerte correlación entre pace y dribbling con el potencial de los jugadores, especialmente para aquellos en posiciones de banda. Esto sugiere que la velocidad y la habilidad en el regate son atributos cruciales para identificar y desarrollar jugadores jóvenes con alto potencial.

Recomendaciones

1. *Desarrollo de Talento:*

   Enfocar el entrenamiento en atributos físicos como pace y agilidad en las primeras etapas de desarrollo puede ser beneficioso, especialmente para posiciones ofensivas.

   A medida que los jugadores maduran, es importante equilibrar el desarrollo técnico y táctico, priorizando atributos específicos según la posición del jugador.

2. *Scouting y Reclutamiento:*

   Utilizar los datos de distribución de edades y atributos por nacionalidad para identificar nuevas fuentes de talento emergente.

   Considerar tanto los valores promedio como la cantidad total de jugadores al evaluar el valor de mercado de jugadores de diferentes nacionalidades.

3. *Estrategias de Formación:*

   Adaptar las estrategias de formación a las fortalezas naturales de los jugadores, priorizando atributos que son cruciales para sus roles específicos en el campo.

   Monitorear el desarrollo de atributos clave a lo largo del tiempo para ajustar los planes de entrenamiento y maximizar el potencial de los jugadores.

En conclusión, para maximizar la probabilidad de encontrar a un joven con mucho potencial, hay que dar especial enfoque en jugadores de Inglaterra, España, Alemania o Brasil, con edades entre 17 y 21 años. Priorizar aquellos que jueguen en posiciones de banda (LW, RW, LM, RM) y que presenten atributos desarrollados de ritmo, regates y agilidad.

EDA TABLA DINÁMICA

El propósito de esta sección será agregar una base de datos dinámica que se actualice constantemente (cada 6 meses) con valores de mercado reales y duraciones de contratos de jugadores a través de scraping de *TransferMarkt*. Lo anterior para poder lograr una evaluación justa y correcta de los jugadores jóvenes.

Decidimos crear nuestro propio script de scraping, enfocando el análisis en 71 jugadores con un potencial igual o mayor a 89 en el juego, de los cuales analizamos sus datos reales en la página de *TransferMarkt*.

Resultados Obtenidos

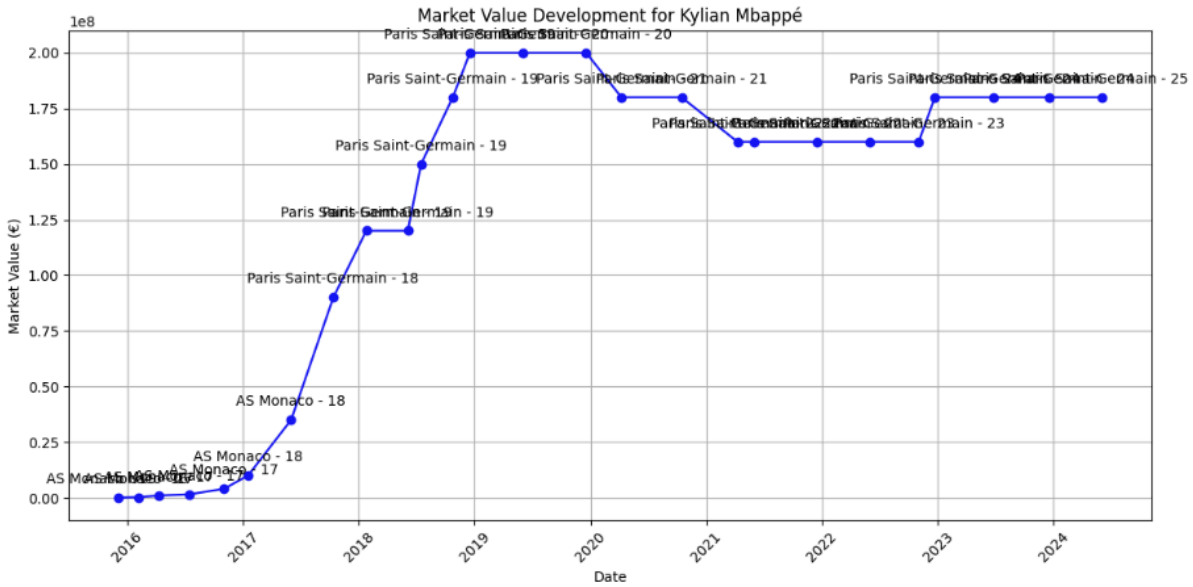
Generamos gráficas detalladas (usando el código a continuación) del historial de valor de mercado de los 10 jugadores con mayor potencial. Estas gráficas muestran la evolución de su valor desde sus inicios en el fútbol hasta la actualidad, permitiéndonos visualizar tendencias y cambios a lo largo del tiempo. Toda la información sacada de aqui es guardada en una tabla de *BigQuery* para tenerla almacenada y poder consultarla más tarde de ser necesario.

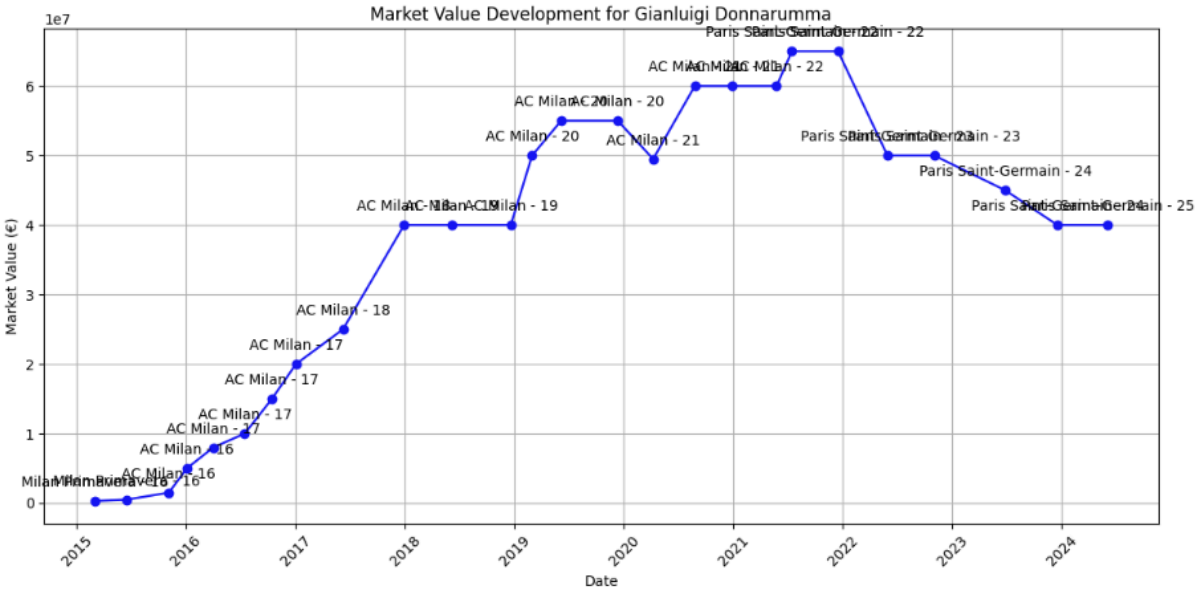
Python
```python
import numpy as np
player_names =[]
player_values = []
player_contract_expiry_dates = []
market_value_last_update = []
dates = []
counter = 0
def get_player_info(url):
 global counter
 player_id = url.split('/')[-1]
 response = requests.get(url,headers=headers)
 if int(response.status_code) == 200:
   soup = BeautifulSoup(response.content, "html.parser")
```

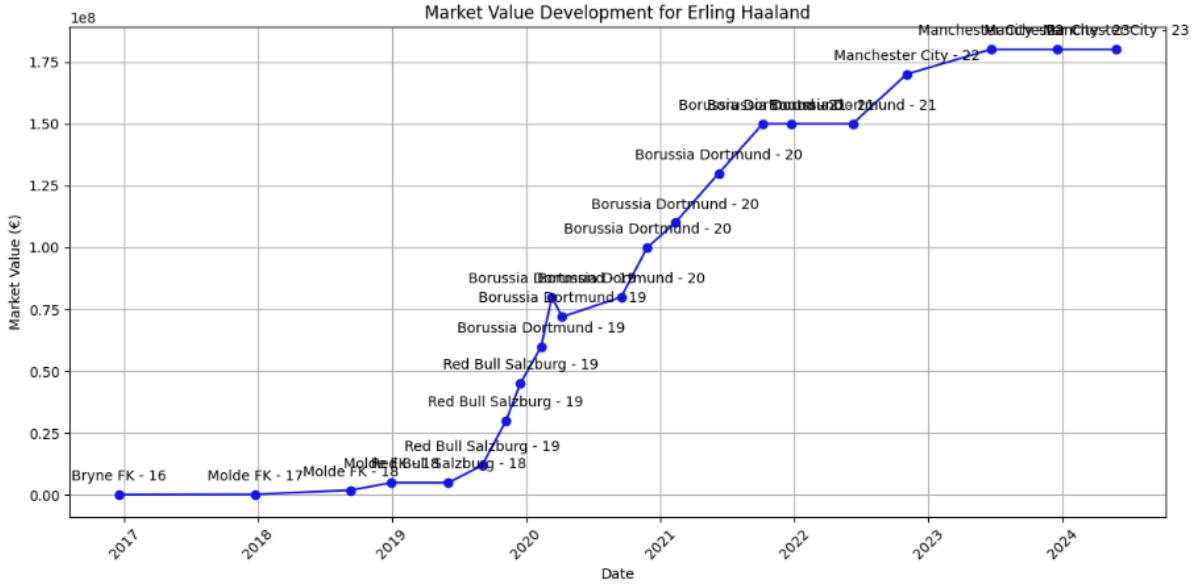
```
    player_name =
soup.select_one('h1[class="data-header__headline-wrapper"]').text
.split('\n')[-1].strip()
    player_contract_expiry = re.search("Contract expires: (.*)",
soup.text)
    if player_contract_expiry != None:
      player_contract_expiry = player_contract_expiry.group(1)
    else:
      player_contract_expiry = np.nan
    market_value = re.search("€(.*)",soup.text).group(0).split("
")[0]
    market_value_date=re.search("€(.*)",soup.text).group(0)
    response = requests.get(

f'https://www.transfermarkt.us/ceapi/marketValueDevelopment/graph
/{player_id}',
      headers=headers
    )
    last_update_MV = extract_date(market_value_date)
    market_value_last_update.append(last_update_MV)
    player_names.append(player_name)
    player_values.append(market_value)
    player_contract_expiry_dates.append(player_contract_expiry)
    current_timestamp = datetime.now()
    date = current_timestamp.strftime('%Y-%m-%d')
    dates.append(date)
    print(f"Player name: {player_name}")
    if counter < 10:
      plot_market_value_hist(response, player_name)
      counter += 1

    print(f"Current market value: {market_value}")
    print(f"Contract expiry date: {player_contract_expiry}\n")
  else:
```

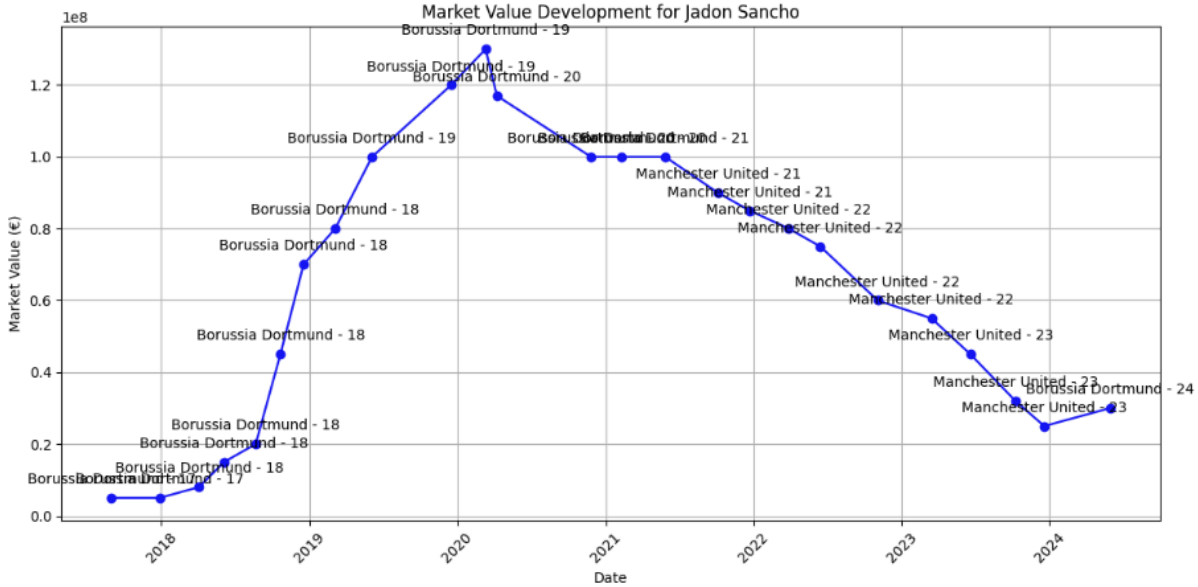
```
    print("Hubo un error al intentar obtener los datos del jugador.")
```

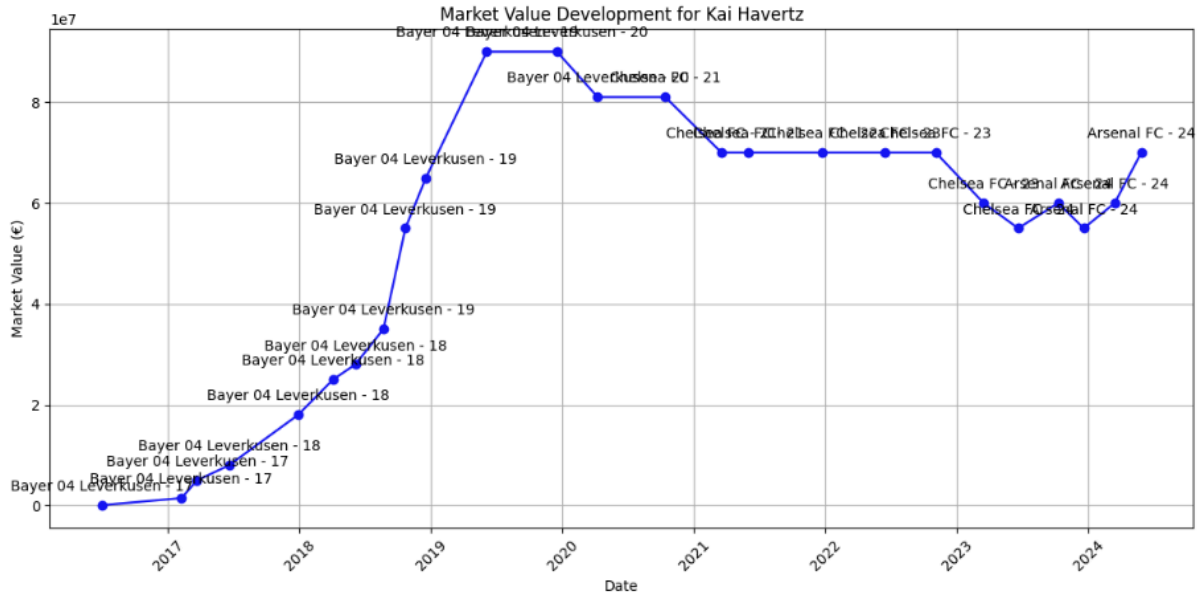
Hasta ahora, hemos logrado combinar datos estáticos de FIFA con valores de mercado reales obtenidos mediante scraping de *TransferMarkt*, permitiéndonos hacer análisis más precisos sobre la inversión en jugadores jóvenes.

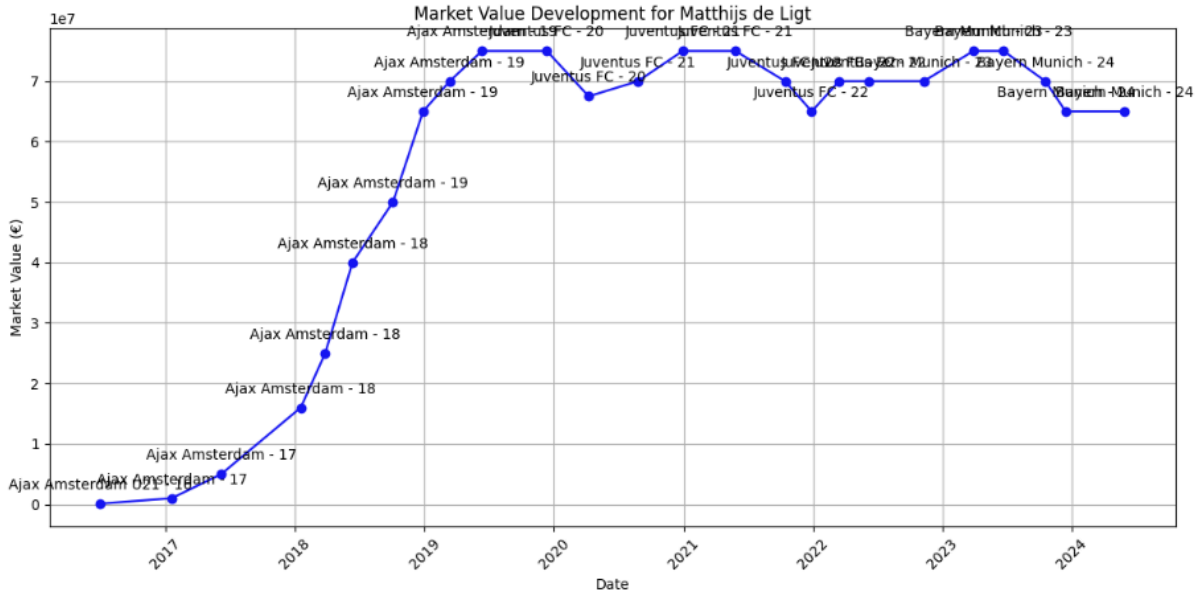


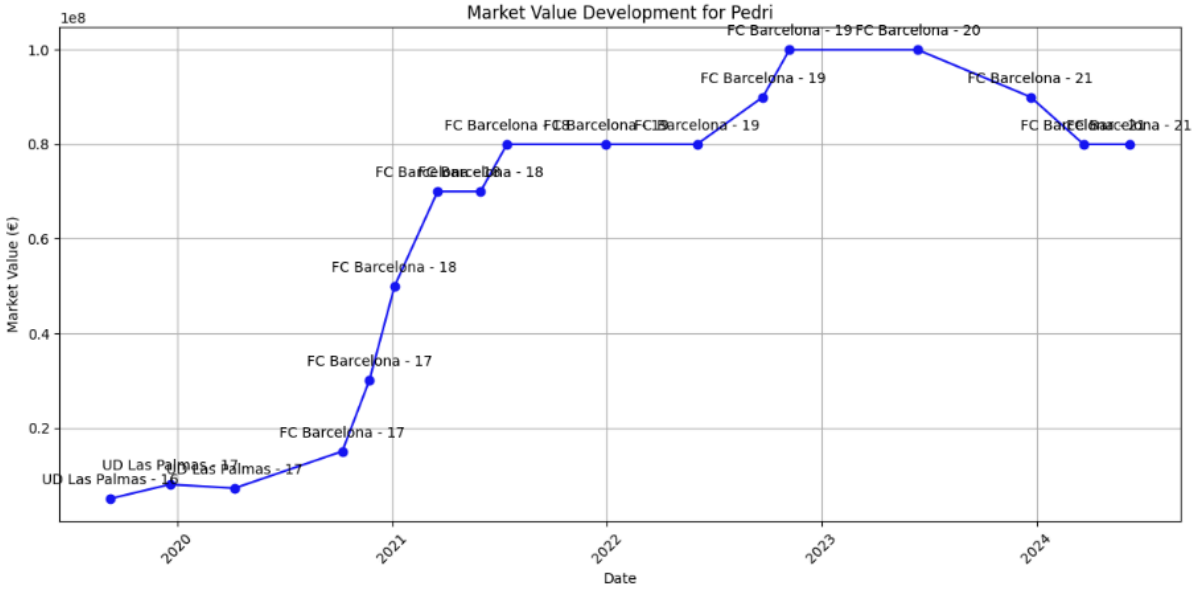


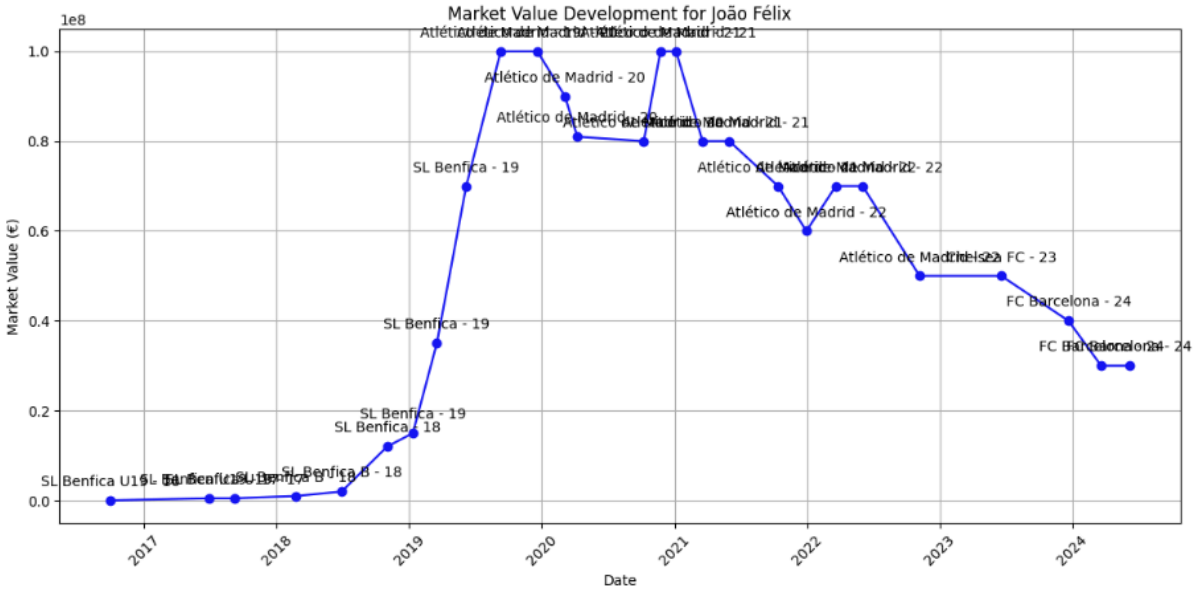

Market Value Development for Erling Haaland
1e8
Market Value (€)
Date
Bryne FK - 16
Molde FK - 17
Molde FK - 18
Red Bull Salzburg - 18
Red Bull Salzburg - 19
Borussia Dortmund - 19
Borussia Dortmund - 20
Borussia Dortmund - 21
Manchester City - 22
Manchester City - 23

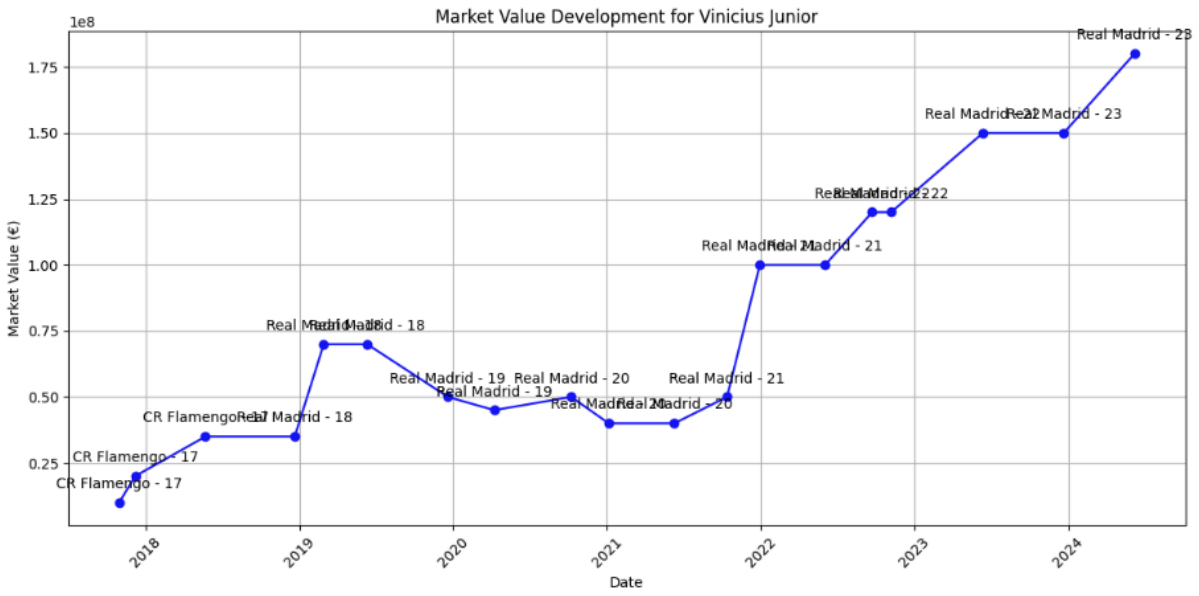

Market Value Development for Jadon Sancho
1e8
Market Value (€)
Date
Borussia Dortmund - 17
Borussia Dortmund - 18
Borussia Dortmund - 19
Borussia Dortmund - 20
Borussia Dortmund - 21
Manchester United - 21
Manchester United - 22
Manchester United - 23
Borussia Dortmund - 24

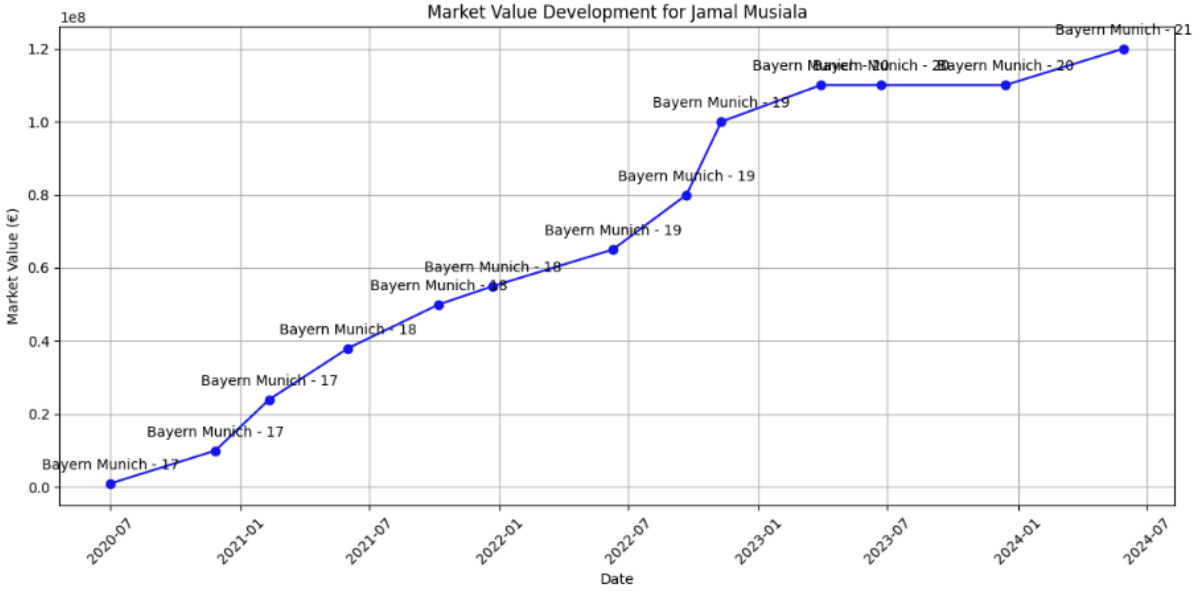

Market Value Development for Kai Havertz
1e7
Market Value (€)
Date
Bayer 04 Leverkusen - 17
Bayer 04 Leverkusen - 18
Bayer 04 Leverkusen - 19
Bayer 04 Leverkusen - 20
Bayer 04 Leverkusen - 21
Chelsea FC - 23
Arsenal FC - 24


Market Value Development for Matthijs de Ligt
1e7
Market Value (€)
Date
Ajax Amsterdam - 17
Ajax Amsterdam - 18
Ajax Amsterdam - 19
Juventus FC - 20
Juventus FC - 21
Juventus FC - 22
Bayern Munich - 23
Bayern Munich - 24


Market Value Development for Pedri
1e8
Market Value (€)
Date
UD Las Palmas - 17
FC Barcelona - 17
FC Barcelona - 18
FC Barcelona - 19
FC Barcelona - 20
FC Barcelona - 21


Market Value Development for João Félix
1e8
Market Value (€)
Date
SL Benfica - 18
SL Benfica - 19
Atlético de Madrid - 20
Atlético de Madrid - 21
Atlético de Madrid - 22
FC Barcelona - 23
FC Barcelona - 24





En nuestro análisis, generamos varias gráficas que representan el historial de valor de mercado de jugadores con alto potencial, como Kylian Mbappé, Gianluigi Donnarumma y Erling Haaland. El objetivo de estas gráficas es identificar patrones tempranos en sus carreras que puedan anticipar un “boom” o explosión en su valor de mercado. Este análisis nos permitirá hacer inversiones estratégicas en jugadores jóvenes antes de que su valor se dispare.

Análisis de Patrones:

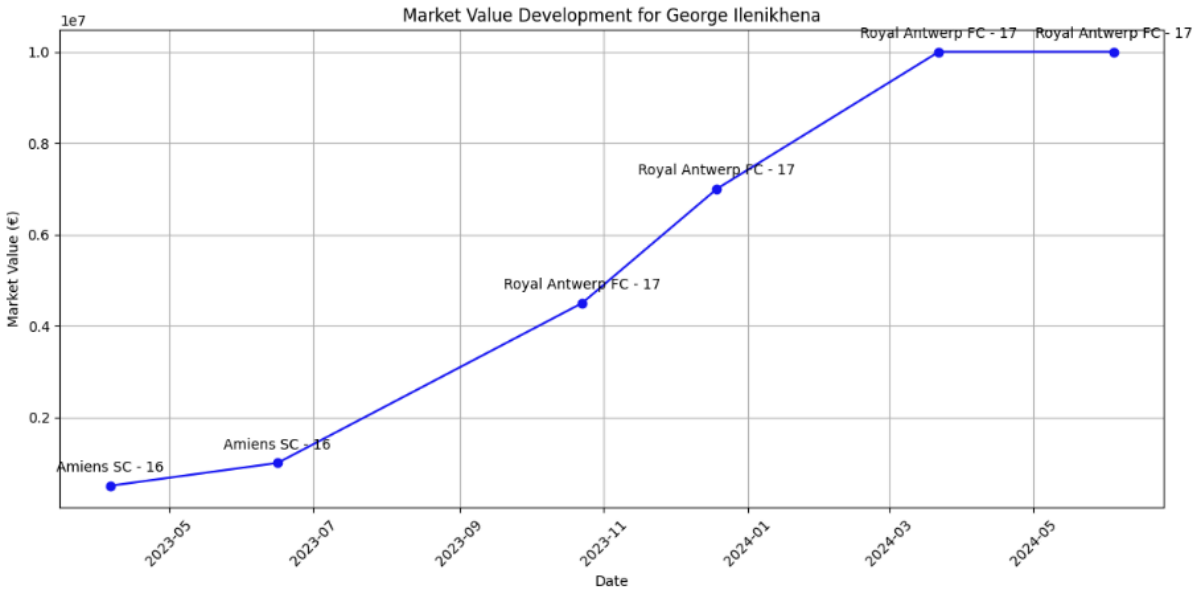
1. <u>Selección de Jugadores:</u>

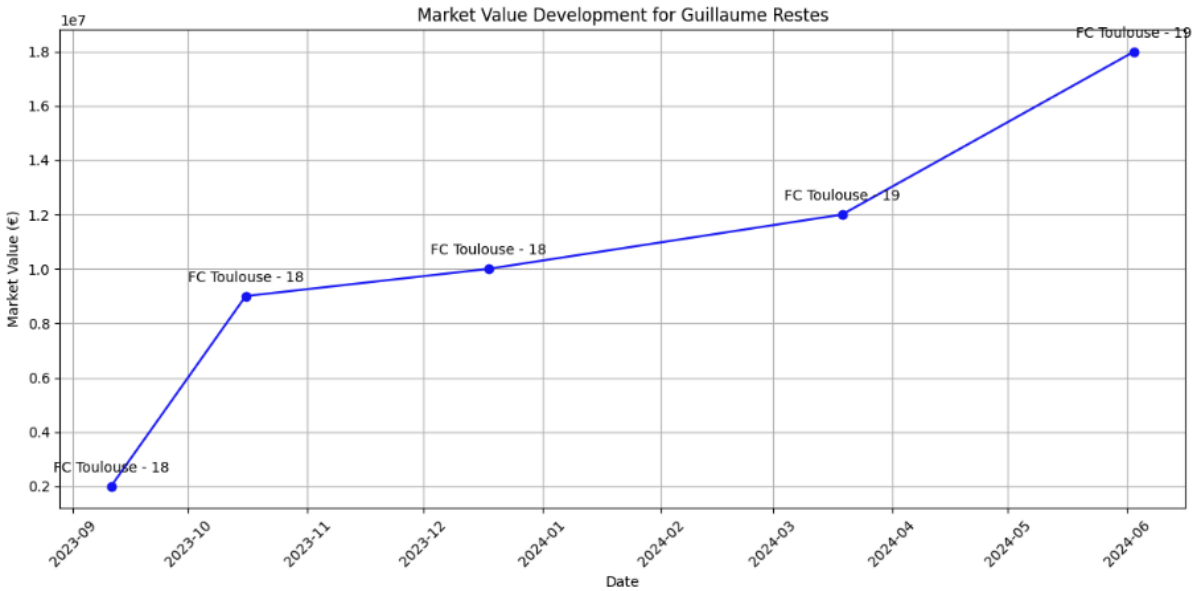
   a. Elegimos 10 jugadores que actualmente son considerados de élite y que han experimentado una explosión significativa en su valor de mercado.
   b. Analizamos sus valores de mercado desde el inicio de sus carreras hasta el presente.
2. Patrón Detectado:
   a. Observamos que, al principio de sus carreras, estos jugadores muestran un crecimiento constante en su valor de mercado.
   b. Este crecimiento inicial es modesto pero constante.
   c. Llega un punto en sus primeras etapas donde el crecimiento se vuelve exponencial, con un aumento rápido y significativo en un corto periodo de tiempo.
3. Objetivo del Análisis:
   a. Nuestro objetivo es anticipar esta fase de crecimiento exponencial.
   b. Identificar jugadores jóvenes que presentan patrones de valor de mercado similares al principio de sus carreras.

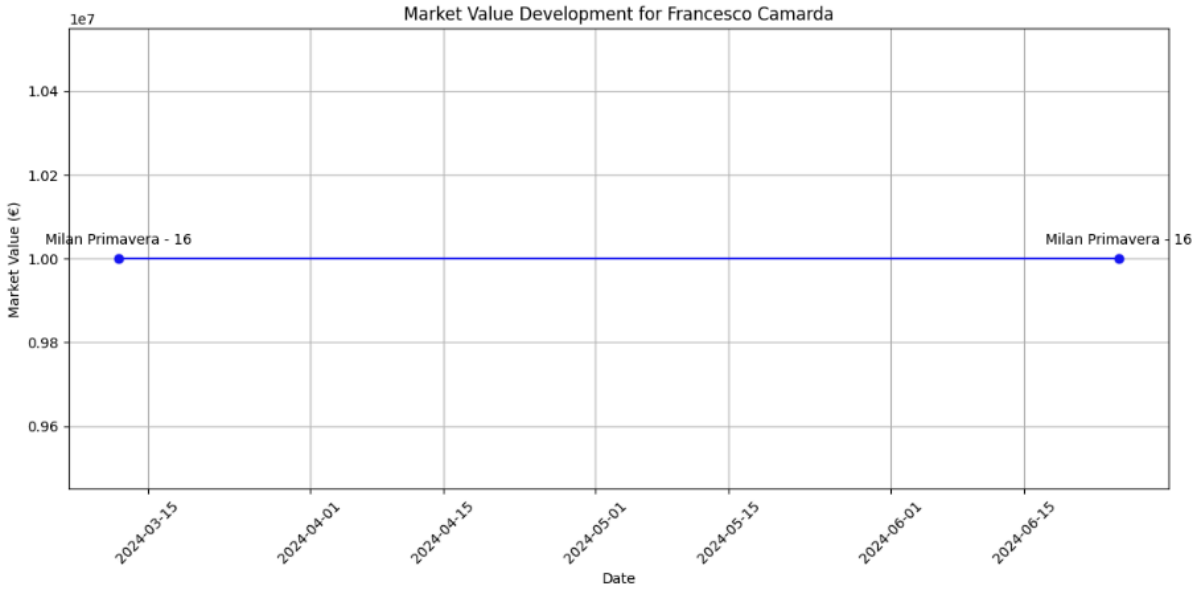
Aplicación del Análisis:

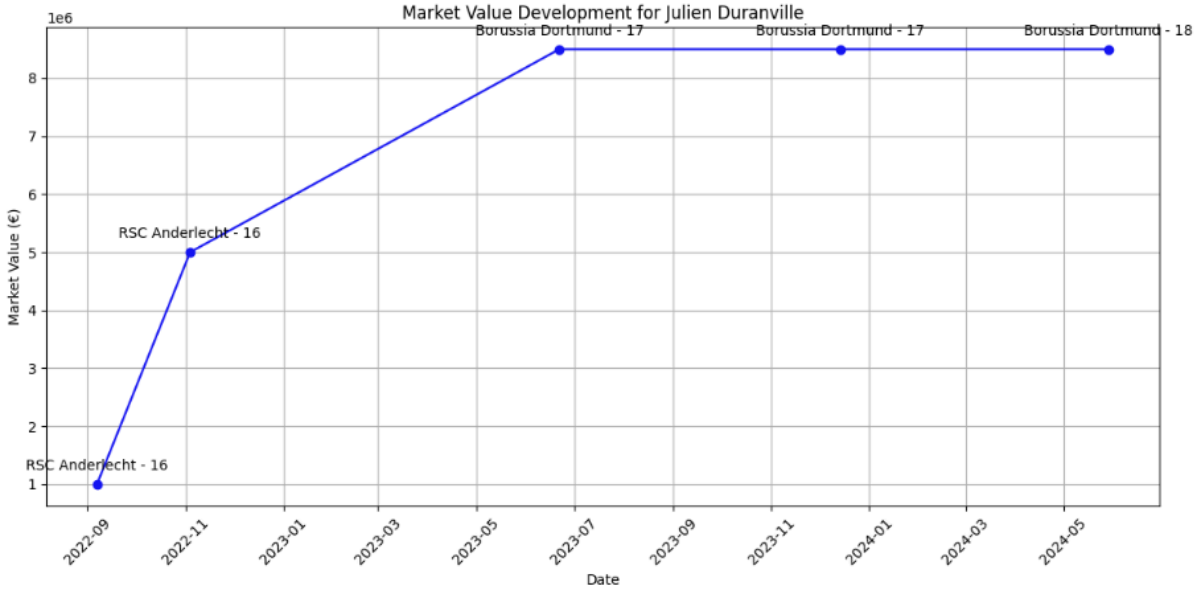
1. Selección de Nuevos Talentos:
   a. Con base en los patrones identificados, seleccionamos otros 10 jugadores menores de 19 años que muestran un comportamiento similar en sus valores de mercado.
   b. Creemos que estos jugadores están en el umbral de una explosión en su valor de mercado similar a la de los jugadores ya establecidos.
2. Inversión Estratégica:
   a. Al anticipar esta explosión, podemos hacer inversiones estratégicas en estos jugadores jóvenes antes de que su valor se dispare.
   b. Esto permitirá maximizar el retorno de inversión y asegurar la adquisición de talentos emergentes.

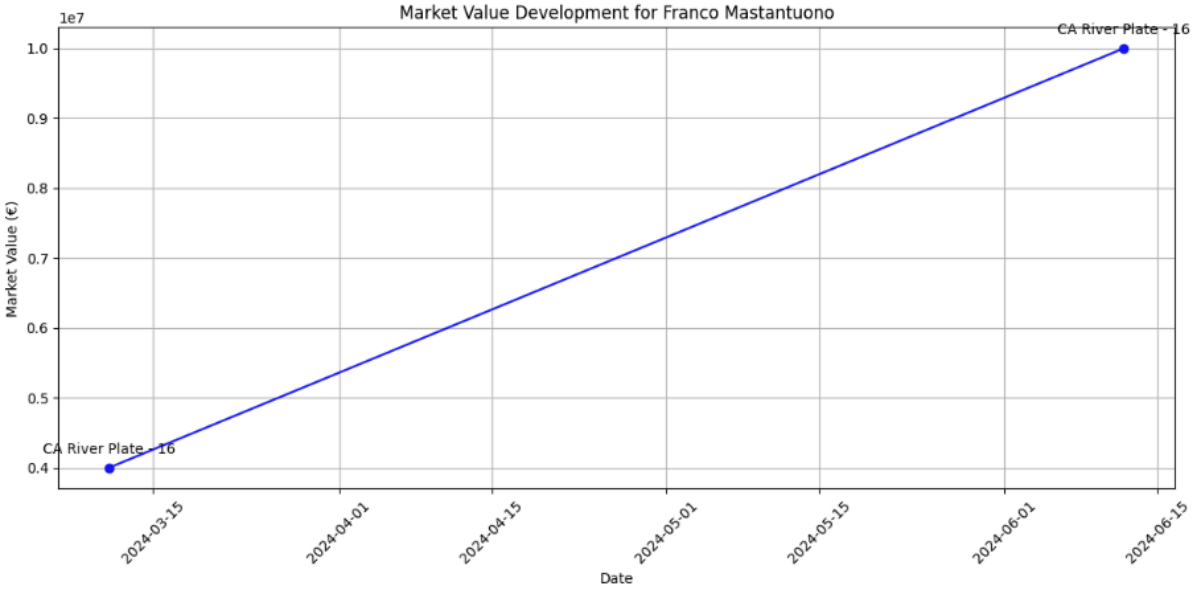
Este enfoque nos brinda una ventaja competitiva al identificar y anticipar el crecimiento exponencial en el valor de mercado de jugadores jóvenes.

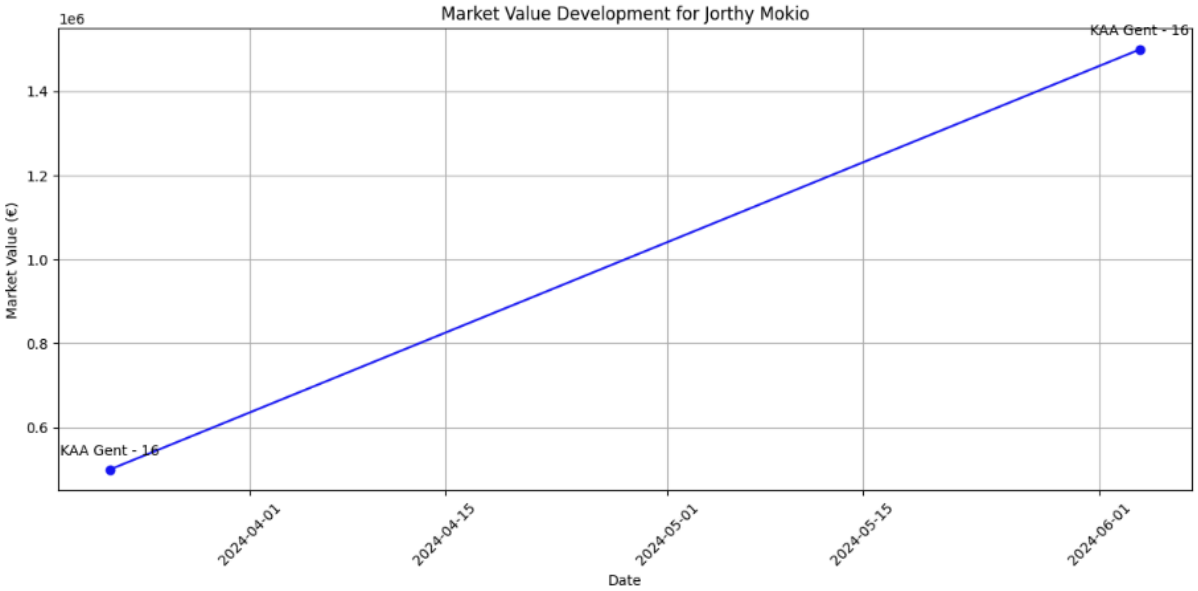

Market Value Development for George Ilenikhena
1e7
Market Value (€)
Date
Amiens SC - 16
Amiens SC - 16
Royal Antwerp FC - 17
Royal Antwerp FC - 17
Royal Antwerp FC - 17
Royal Antwerp FC - 17

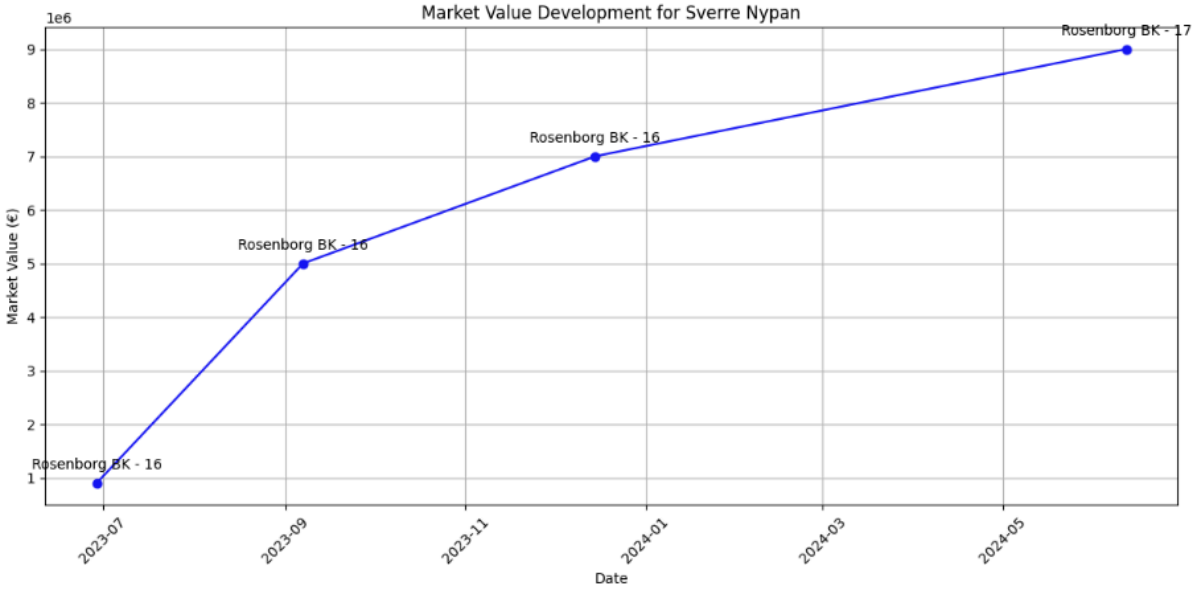

Market Value Development for Guillaume Restes
1e7
Market Value (€)
Date
FC Toulouse - 18
FC Toulouse - 18
FC Toulouse - 18
FC Toulouse - 19
FC Toulouse - 19

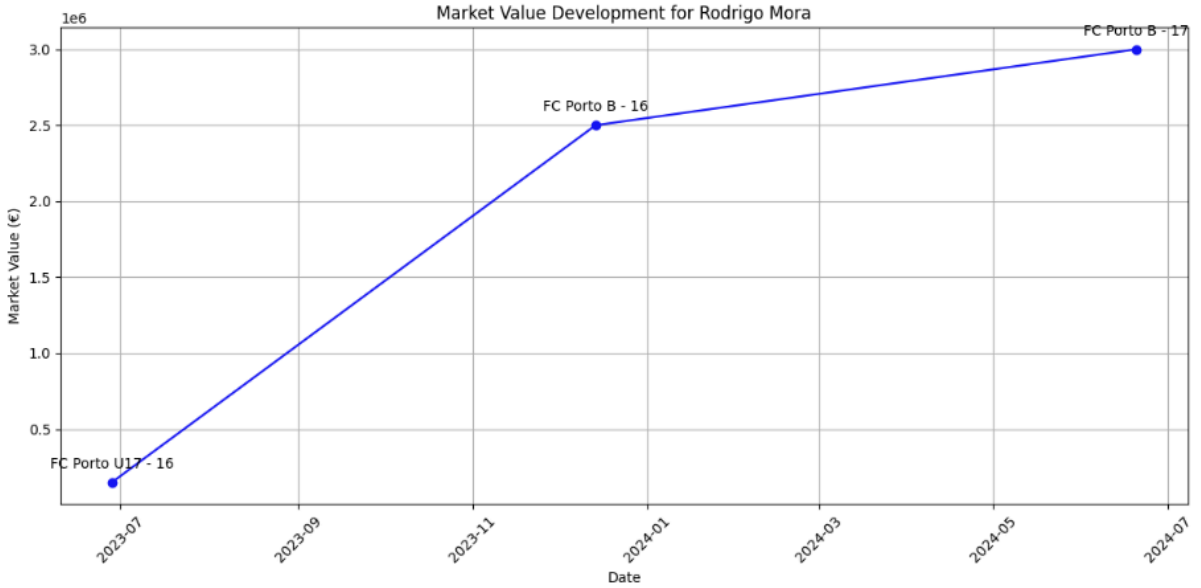

Market Value Development for Francesco Camarda
1e7
Market Value (€)
Date
Milan Primavera - 16
Milan Primavera - 16

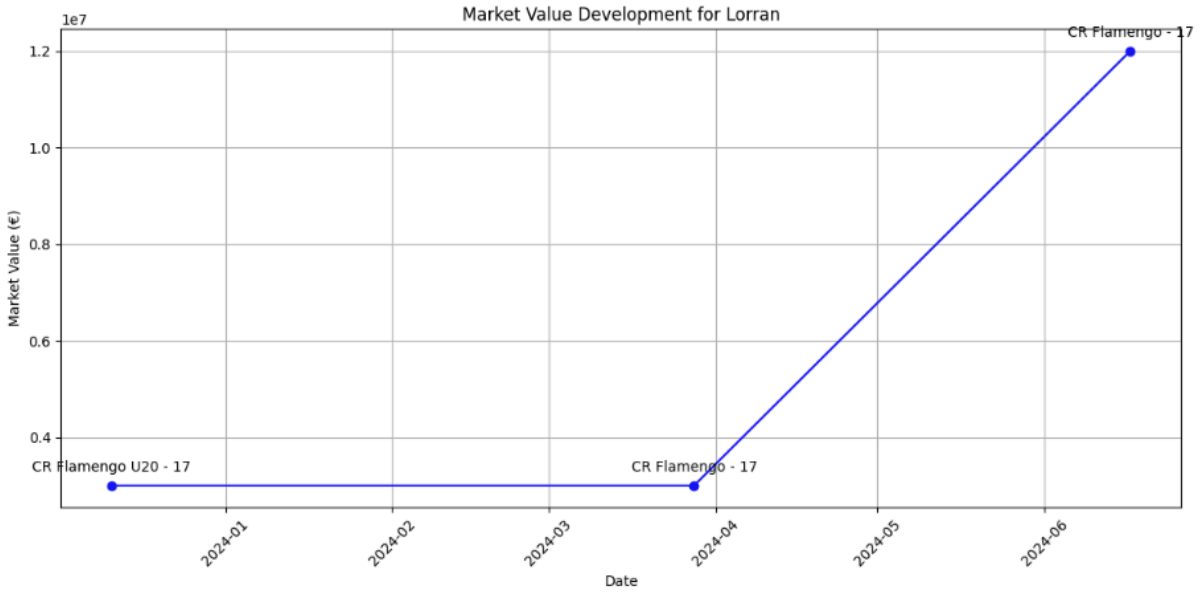

Market Value Development for Julien Duranville
1e6
Borussia Dortmund - 17
Borussia Dortmund - 17
Borussia Dortmund - 18
RSC Anderlecht - 16
RSC Anderlecht - 16
Market Value (€)
Date
2022-09
2022-11
2023-01
2023-03
2023-05
2023-07
2023-09
2023-11
2024-01
2024-03
2024-05


Market Value Development for Franco Mastantuono
1e7
CA River Plate - 16
CA River Plate - 16
Market Value (€)
Date
2024-03-15
2024-04-01
2024-04-15
2024-05-01
2024-05-15
2024-06-01
2024-06-15


Market Value Development for Jorthy Mokio
1e6
KAA Gent - 16
KAA Gent - 16
Market Value (€)
Date
2024-04-01
2024-04-15
2024-05-01
2024-05-15
2024-06-01


Market Value Development for Sverre Nypan
1e6
Market Value (€)
Date
Rosenborg BK - 16
Rosenborg BK - 16
Rosenborg BK - 16
Rosenborg BK - 17
2023-07
2023-09
2023-11
2024-01
2024-03
2024-05


Market Value Development for Rodrigo Mora
1e6
Market Value (€)
Date
FC Porto U17 - 16
FC Porto B - 16
FC Porto B - 17
2023-07
2023-09
2023-11
2024-01
2024-03
2024-05
2024-07


Market Value Development for Lorran
1e7
Market Value (€)
Date
CR Flamengo U20 - 17
CR Flamengo - 17
CR Flamengo - 17
2024-01
2024-02
2024-03
2024-04
2024-05
2024-06

Evaluación del Impacto de los Torneos Internacionales en el Valor de Mercado de Jugadores Jóvenes

Posteriormente, decidimos tomar el enfoque de evaluar el impacto de los torneos internacionales como la Eurocopa, la Copa América y el Mundial en el valor de mercado de los jugadores. Creemos que estos eventos globales pueden aumentar la visibilidad y el valor de los jugadores, lo que nos permitiría invertir en jóvenes talentos antes de que sus precios se disparen, obteniendo así un mayor beneficio.

Nuestro equipo parte de la hipótesis de que el desempeño en grandes torneos internacionales, como la Eurocopa, la Copa América o el Mundial, puede influir significativamente en el valor de mercado de los jugadores. Esta hipótesis se basa en la observación de que estos torneos atraen una atención global considerable, destacando el talento y las habilidades de los jugadores en una plataforma internacional.

Objetivo del Análisis:

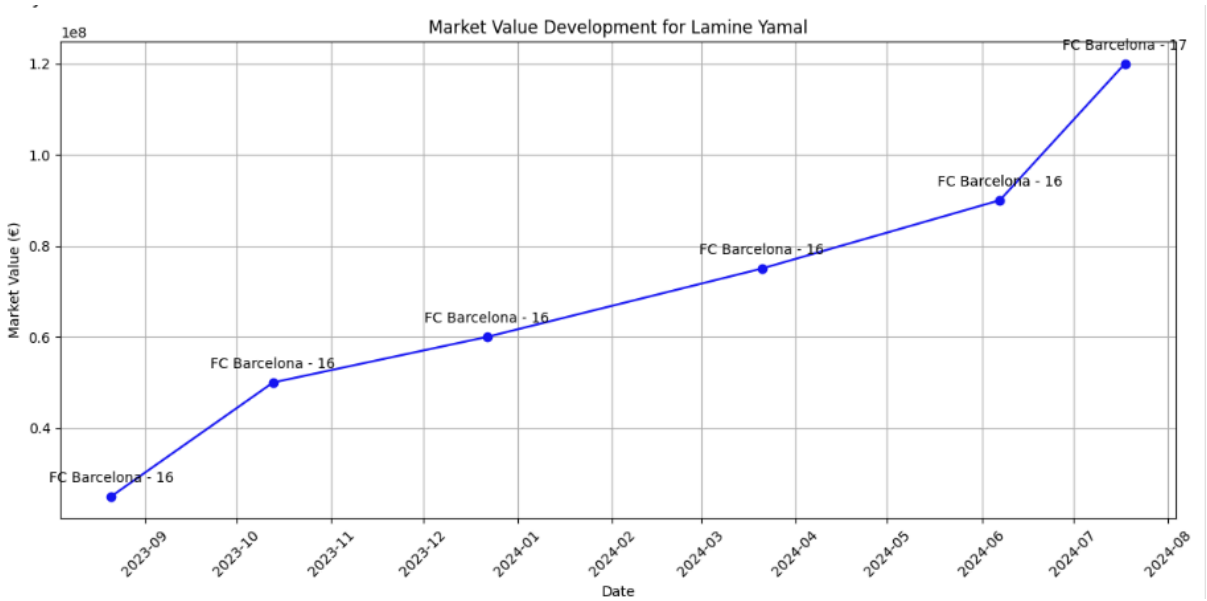
El objetivo de este análisis es investigar cómo el rendimiento de los jugadores en la Eurocopa actual afecta sus valores de mercado. Para ello, compararemos los valores de mercado de los jugadores antes y después de estos torneos, utilizando datos obtenidos mediante scraping de TransferMarkt.

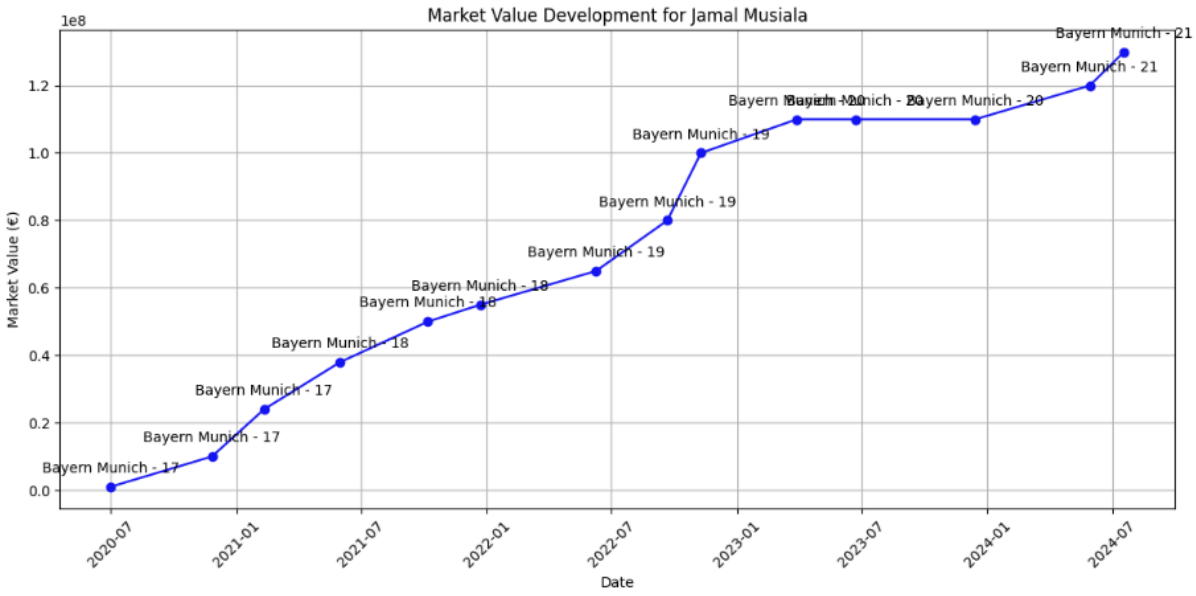
Método:

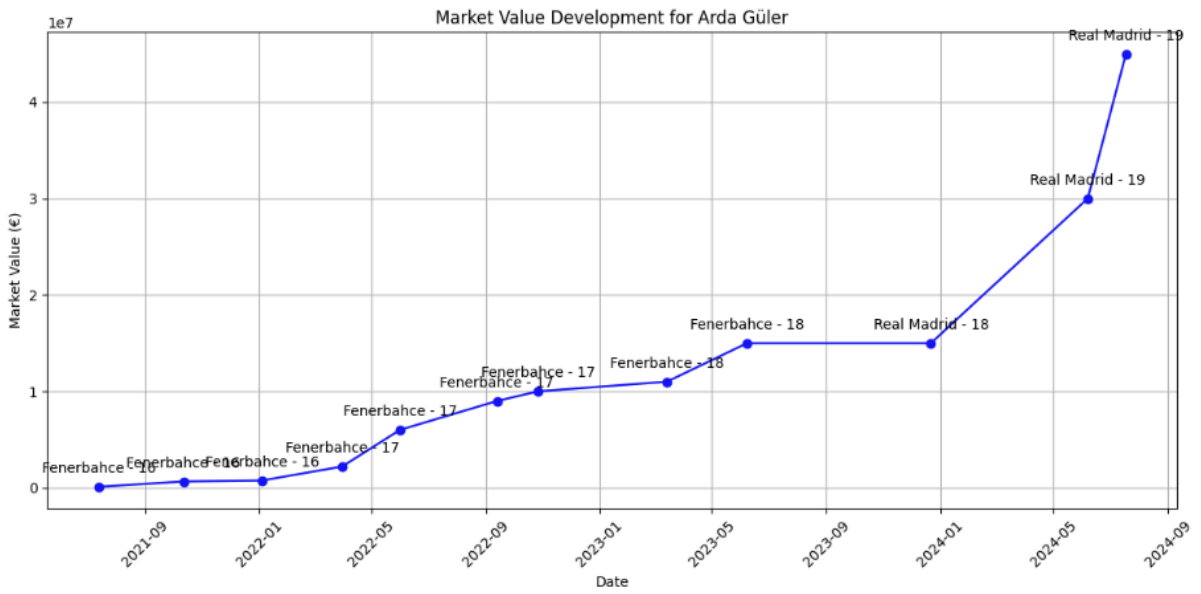
1. Selección de Jugadores:
    a. Identificaremos a los jugadores que participaron en la Eurocopa actual.
    b. Recopilaremos datos sobre sus valores de mercado antes del inicio de los torneos.
2. Recopilación de Datos Posteriores a los Torneos:
    a. Una vez concluidos los torneos, recopilaremos nuevamente los datos de valor de mercado de los mismos jugadores.
3. Análisis Comparativo:

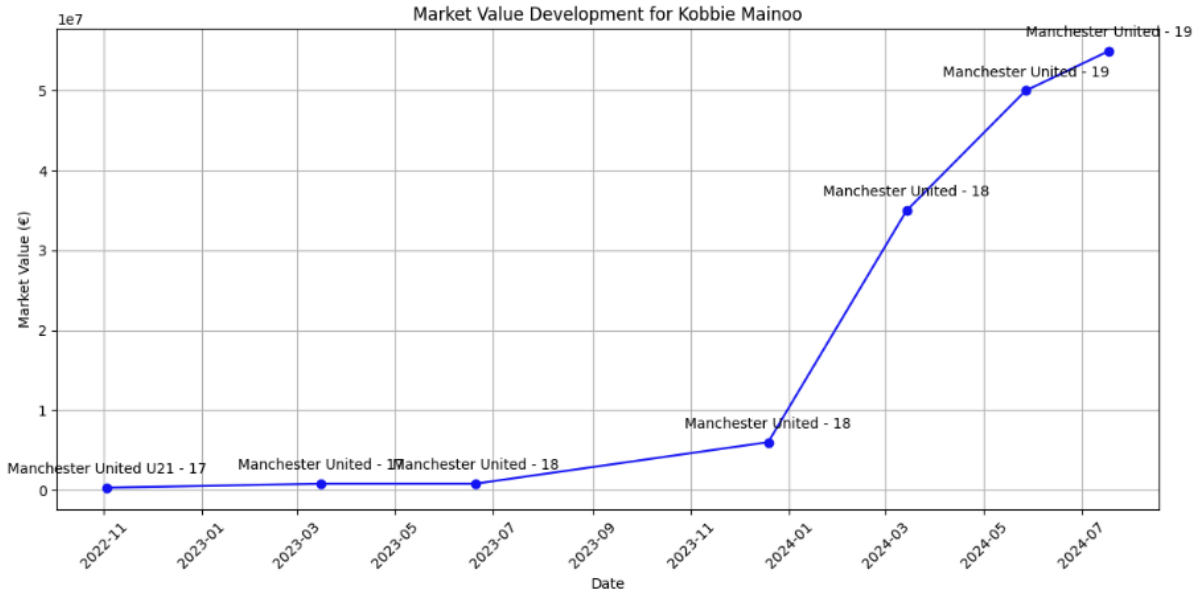
a. Compararemos los valores de mercado antes y después de los torneos para identificar cambios significativos.
b. Generamos gráficas que ilustran la evolución del valor de mercado de los jugadores seleccionados, destacando aquellos con aumentos o disminuciones notables.

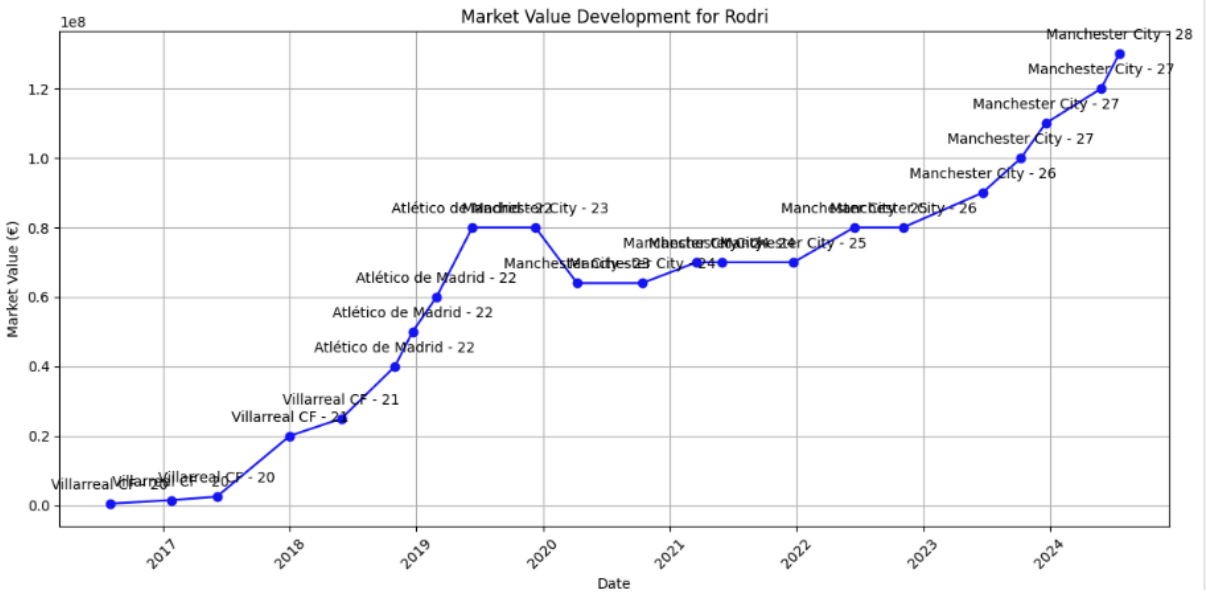
Esperamos que este análisis proporcione una comprensión más profunda de cómo los grandes torneos internacionales pueden impactar el valor de mercado de los jugadores, ofreciendo insights valiosos para scouts, clubes y analistas del fútbol.

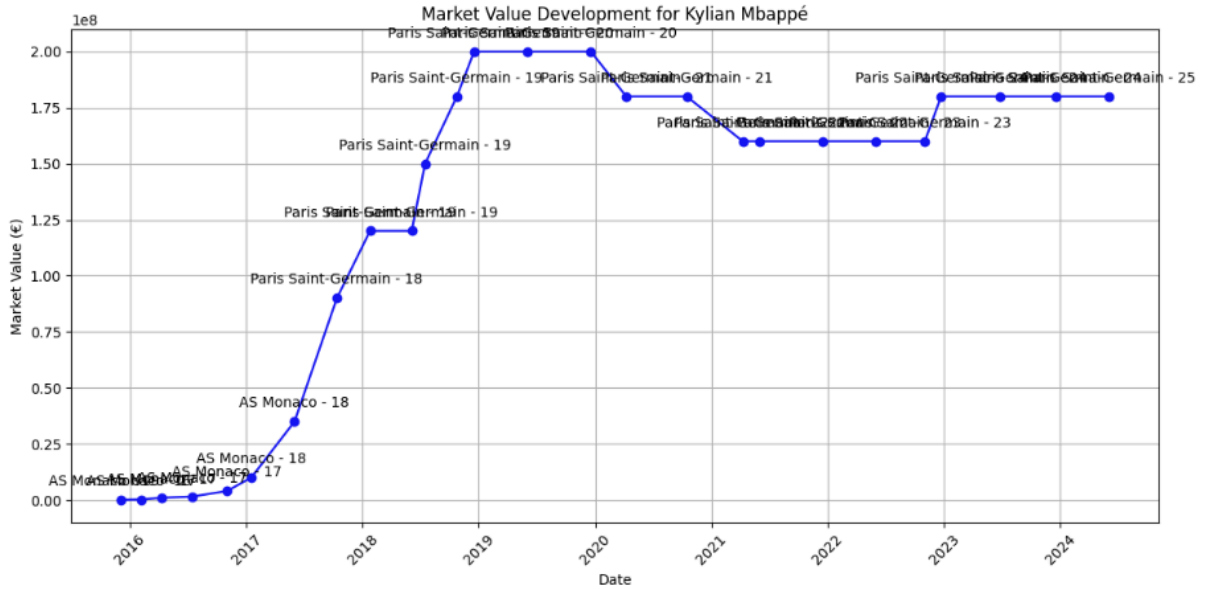


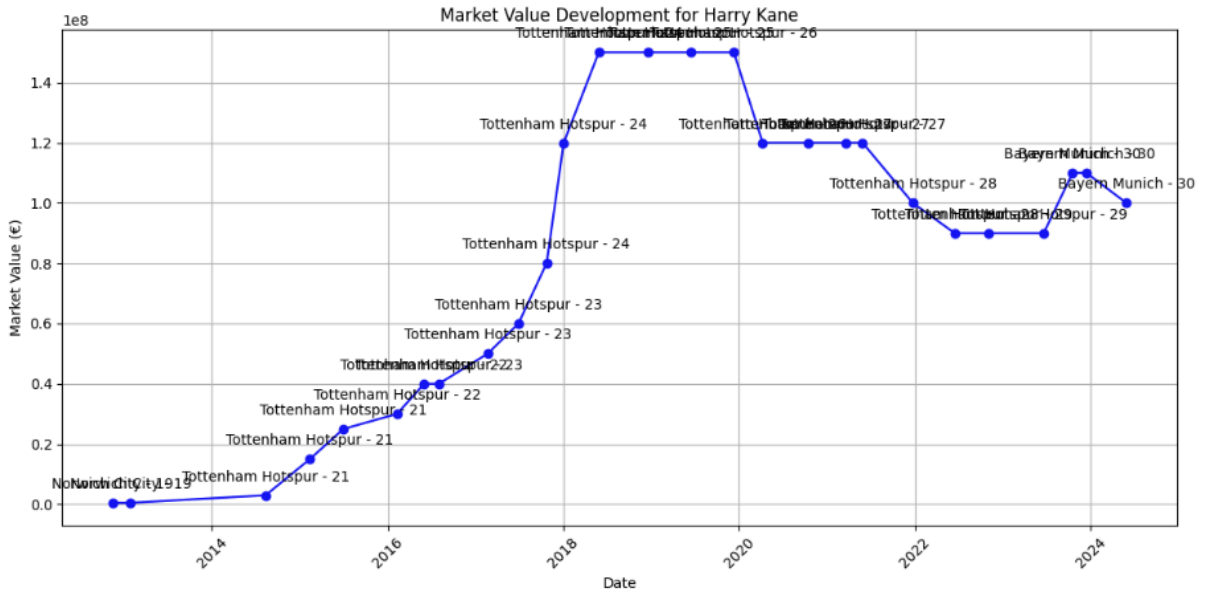

Market Value Development for Jamal Musiala
1e8
Market Value (€)
Date
Bayern Munich - 17
Bayern Munich - 17
Bayern Munich - 17
Bayern Munich - 18
Bayern Munich - 18
Bayern Munich - 18
Bayern Munich - 19
Bayern Munich - 19
Bayern Munich - 19
Bayern Munich - 20
Bayern Munich - 20
Bayern Munich - 21
Bayern Munich - 21


Market Value Development for Arda Güler
1e7
Market Value (€)
Date
Fenerbahce - 16
Fenerbahce - 16
Fenerbahce - 16
Fenerbahce - 17
Fenerbahce - 17
Fenerbahce - 17
Fenerbahce - 17
Fenerbahce - 18
Fenerbahce - 18
Real Madrid - 18
Real Madrid - 19
Real Madrid - 19


Market Value Development for Kobbie Mainoo
1e7
Market Value (€)
Date
Manchester United U21 - 17
Manchester United - 17
Manchester United - 18
Manchester United - 18
Manchester United - 18
Manchester United - 19
Manchester United - 19


Market Value Development for Rodri
1e8
Market Value (€)
Date
Villarreal CF - 20
Villarreal CF - 21
Atlético de Madrid - 22
Manchester City - 23
Manchester City - 25
Manchester City - 26
Manchester City - 27
Manchester City - 28


Market Value Development for Kylian Mbappé
1e8
Market Value (€)
Date
AS Monaco - 18
Paris Saint-Germain - 18
Paris Saint-Germain - 19
Paris Saint-Germain - 20
Paris Saint-Germain - 21
Paris Saint-Germain - 25


Market Value Development for Harry Kane
1e8
Market Value (€)
Date
Tottenham Hotspur - 21
Tottenham Hotspur - 22
Tottenham Hotspur - 23
Tottenham Hotspur - 24
Tottenham Hotspur - 28
Bayern Munich - 30

CONCLUSIÓN

Nuestros resultados confirman que la participación en torneos internacionales influye en el valor de mercado de los jugadores. Más aún, descubrimos que esta influencia es exclusivamente positiva. Al analizar las gráficas de jugadores que se desempeñaron bien en estos torneos, observamos que su valor de mercado aumentó en promedio unos 10 millones después del evento. En contraste, los jugadores que tuvieron un desempeño deficiente no experimentaron ninguna disminución en su valor de mercado. Esto demuestra que la presencia en torneos internacionales efectivamente afecta el valor de los jugadores, pero únicamente de manera positiva.

Recomendaciones:

Recomendamos invertir en jugadores jóvenes antes de grandes torneos internacionales como la Eurocopa, la Copa América y el Mundial. Nuestro análisis ha demostrado que, en caso de que los jugadores se desempeñen bien en estos torneos, su valor de mercado aumentará significativamente. Además, incluso si su desempeño es deficiente, su valor no disminuirá. Por lo tanto, esta estrategia de inversión presenta un riesgo mínimo con un alto potencial de retorno.

TROUBLESHOOTING

Error en los Worker Nodes:

- **Región**: US West 1
- **Tipo de nodo inicial**: N1
- **Problemas**:
  - No había suficiente espacio SSD
  - No había suficientes núcleos
  - No había suficiente RAM
- **Solución**:
  - Cambiar el tipo de nodos de N2 a N1
  - Cambiar la configuración de almacenamiento:
    - Nodo principal de 500 GB a 100 GB
    - Tres nodos trabajadores de 500 GB cada uno a 50 GB cada uno (total de 250 GB)

Error en la conexión de Hive con el bucket:

- **Descripción del error**:
  - No se incluyó el prefijo `gs://` al escribir la clave de Hive en el path.
- **Impacto**: Error al intentar inicializar el cluster.
- **Solución**: Corregir la configuración agregando `gs://` al valor de la clave de Hive.

Error al intentar crear el clúster:

- **Descripción del error**:
  - No se podía acceder al archivo de acciones de inicialización.
- **Soluciones intentadas**:
  - Abrir todos los puertos para no restringir ninguna conexión.
  - Configurar el firewall.
- **Solución final**:
  - Cargar el dataset directamente en el Hadoop de la máquina virtual para poder trabajar con los datos.

```
2024-07-09 22:15:43.000  Initialization failed. Enabling Diagnostic mode
com.google.cloud.hadoop.services.agent.AgentException: Initialization action failed. Failed action
'gs://bucket-proyecto-datos-2/cloud-sql-proxy.sh', see output in: gs://dataproc-staging-us-west1-
717228848790-90tdn4ml/google-cloud-dataproc-metainfo/f3a7c81d-c26c-4ee8-827c-386ca4b63883/proyecto-datos-
m/dataproc-initialization-script-0_output (TASK_FAILED)
        at com.google.cloud.hadoop.services.agent.AgentException$Builder.build(AgentException.java:83)
        at
com.google.cloud.hadoop.services.agent.AbstractAgentRunner.extractErrorsAndThrowExceptionIfFailure(AbstractA
gentRunner.java:390)
        at
com.google.cloud.hadoop.services.agent.AbstractAgentRunner.runCustomInitializationActionsIfFirstRun(Abstract
AgentRunner.java:362)
        at com.google.cloud.hadoop.services.agent.MasterAgentRunner.initialize(MasterAgentRunner.java:175)
        at com.google.cloud.hadoop.services.agent.AbstractAgentRunner.start(AbstractAgentRunner.java:132)
        at com.google.cloud.hadoop.services.agent.MasterAgentRunner.start(MasterAgentRunner.java:43)
        at com.google.cloud.hadoop.services.agent.AgentMain.lambda$startAgent$0(AgentMain.java:247)
        at com.google.cloud.hadoop.services.agent.AgentStatusReporter.runWith(AgentStatusReporter.java:49)
        at com.google.cloud.hadoop.services.agent.AgentMain.startAgent(AgentMain.java:243)
        at com.google.cloud.hadoop.services.agent.AgentMain.boot(AgentMain.java:106)
        at com.google.cloud.hadoop.services.agent.AgentMain.main(AgentMain.java:84)
```

```
google-cloud-dataproc-metainfo_41ce267a-a46a-48f5-bf2e-c69902792789_cluster-proyecto-m_dataproc-initialization-script-0_...
ERROR: (gcloud.sql.instances.describe) HTTPError 400: Project specified in the request is invalid.
[2024-07-11T01:10:10+0000] [cluster-proyecto-m]: INFO: Unable to describe metastore instance
Service Hive server2 is not enabled
[2024-07-11T01:10:10+0000] [cluster-proyecto-m]: INFO: Stopping Hive metastore ...
[2024-07-11T01:10:11+0000] [cluster-proyecto-m]: INFO: Hive metastore stopped
[2024-07-11T01:10:11+0000] [cluster-proyecto-m]: INFO: Stopping and disabling mysql.service ...
Removed /etc/systemd/system/multi-user.target.wants/mysql.service.
[2024-07-11T01:10:12+0000] [cluster-proyecto-m]: INFO: mysql.service stopped and disabled
Installing Cloud SQL Proxy ...
2024-07-11 01:10:12 URL:https://dl.google.com/cloudsql/cloud_sql_proxy.linux.amd64 [19921714/19921714] ->
"cloud_sql_proxy.linux.amd64" [1]
[2024-07-11T01:10:12+0000] [cluster-proyecto-m]: INFO: Cloud SQL Proxy installation succeeded
[2024-07-11T01:10:12+0000] [cluster-proyecto-m]: INFO: Starting Cloud SQL proxy ...
Created symlink /etc/systemd/system/multi-user.target.wants/cloud-sql-proxy.service → /lib/systemd/system/cloud-sql-
proxy.service.
[2024-07-11T01:10:12+0000] [cluster-proyecto-m]: INFO: About to run 'nc -zv localhost 3306' with retries...
nc: connect to localhost port 3306 (tcp) failed: Connection refused
[2024-07-11T01:10:12+0000] [cluster-proyecto-m]: INFO: 'nc -zv localhost 3306' attempt 1 failed! Sleeping 1.
nc: connect to localhost port 3306 (tcp) failed: Connection refused
[2024-07-11T01:10:13+0000] [cluster-proyecto-m]: INFO: 'nc -zv localhost 3306' attempt 2 failed! Sleeping 1.
nc: connect to localhost port 3306 (tcp) failed: Connection refused
[2024-07-11T01:10:14+0000] [cluster-proyecto-m]: INFO: 'nc -zv localhost 3306' attempt 3 failed! Sleeping 2.
nc: connect to localhost port 3306 (tcp) failed: Connection refused
[2024-07-11T01:10:16+0000] [cluster-proyecto-m]: INFO: 'nc -zv localhost 3306' attempt 4 failed! Sleeping 3.
nc: connect to localhost port 3306 (tcp) failed: Connection refused
[2024-07-11T01:10:19+0000] [cluster-proyecto-m]: INFO: 'nc -zv localhost 3306' attempt 5 failed! Sleeping 5.
nc: connect to localhost port 3306 (tcp) failed: Connection refused
[2024-07-11T01:10:24+0000] [cluster-proyecto-m]: INFO: 'nc -zv localhost 3306' attempt 6 failed! Sleeping 8.
nc: connect to localhost port 3306 (tcp) failed: Connection refused
[2024-07-11T01:10:32+0000] [cluster-proyecto-m]: INFO: 'nc -zv localhost 3306' attempt 7 failed! Sleeping 13.
nc: connect to localhost port 3306 (tcp) failed: Connection refused
[2024-07-11T01:10:45+0000] [cluster-proyecto-m]: INFO: 'nc -zv localhost 3306' attempt 8 failed! Sleeping 21.
nc: connect to localhost port 3306 (tcp) failed: Connection refused
[2024-07-11T01:11:06+0000] [cluster-proyecto-m]: INFO: 'nc -zv localhost 3306' attempt 9 failed! Sleeping 34.
nc: connect to localhost port 3306 (tcp) failed: Connection refused
[2024-07-11T01:11:40+0000] [cluster-proyecto-m]: INFO: 'nc -zv localhost 3306' attempt 10 failed! Sleeping 55.
nc: connect to localhost port 3306 (tcp) failed: Connection refused
[2024-07-11T01:12:35+0000] [cluster-proyecto-m]: INFO: 'nc -zv localhost 3306' attempt 11 failed! Sleeping 89.
nc: connect to localhost port 3306 (tcp) failed: Connection refused
[2024-07-11T01:14:04+0000] [cluster-proyecto-m]: INFO: 'nc -zv localhost 3306' attempt 12 failed! Sleeping 144.
[2024-07-11T01:16:28+0000] [cluster-proyecto-m]: INFO: Final attempt of 'nc -zv localhost 3306'...
nc: connect to localhost port 3306 (tcp) failed: Connection refused
```

Error al acceder a los datos en Jupyter Notebook desde Hadoop con Pandas:

- **Descripción del error**:
  - Al intentar leer los datos desde Hadoop con Pandas, el sistema se reiniciaba y crasheaba.
- **Solución**: Utilizar PySpark en lugar de Pandas para cargar los datos en Jupyter Notebook.

```
In [ ]: import pandas as pd

        # Lee el archivo CSV desde HDFS
        df = pd.read_csv('hdfs:///user/usuario/datasets/csv/male_players.csv')

        # Ahora puedes trabajar con df, que es tu DataFrame de pandas
        df.head

/opt/conda/miniconda3/lib/python3.10/site-packages/fsspec/implementations/hdfs.py:83: FutureWarning: pyarrow.hdfs.H
adoopFileSystem is deprecated as of 2.0.0, please use pyarrow.fs.HadoopFileSystem instead.
  self.client = HadoopFileSystem(
2024-07-11 04:25:25,963 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using
builtin-java classes where applicable
2024-07-11 04:25:27,368 WARN shortcircuit.DomainSocketFactory: The short-circuit local reads feature cannot be used
because libhadoop cannot be loaded.
```

Error al visualizar los datos del potencial de los jugadores jóvenes:

- **Descripción del error**:
  - Los datos estaban guardados como strings y se necesitaba editarlos como números.
- **Solución**: Convertir los datos de string a int.

```
----> 3 sns.barplot(x='potential', y='short_name', hue='nationality_name', data=young_players_df)
      4 plt.title('Top 10 de Jugadores Jóvenes con Alto Potencial')
      5 plt.xlabel('Potencial')

/opt/conda/miniconda3/lib/python3.10/site-packages/seaborn/categorical.py in barplot(data, x, y, hue, order, hue_or
der, estimator, errorbar, n_boot, units, seed, orient, color, palette, saturation, width, errcolor, errwidth, capsi
ze, dodge, ci, ax, **kwargs)
   2753         estimator = "size"
   2754
-> 2755     plotter = _BarPlotter(x, y, hue, data, order, hue_order,
   2756                           estimator, errorbar, n_boot, units, seed,
   2757                           orient, color, palette, saturation,

/opt/conda/miniconda3/lib/python3.10/site-packages/seaborn/categorical.py in __init__(self, x, y, hue, data, order,
hue_order, estimator, errorbar, n_boot, units, seed, orient, color, palette, saturation, width, errcolor, errwidth,
capsize, dodge)
   1528                 errcolor, errwidth, capsize, dodge):
   1529         """Initialize the plotter."""
-> 1530         self.establish_variables(x, y, hue, data, orient,
   1531                                  order, hue_order, units)
   1532         self.establish_colors(color, palette, saturation)

/opt/conda/miniconda3/lib/python3.10/site-packages/seaborn/categorical.py in establish_variables(self, x, y, hue, d
ata, orient, order, hue_order, units)
    542
    543             # Figure out the plotting orientation
--> 544             orient = infer_orient(
    545                 x, y, orient, require_numeric=self.require_numeric
    546             )

/opt/conda/miniconda3/lib/python3.10/site-packages/seaborn/_oldcore.py in infer_orient(x, y, orient, require_numeri
c)
   1630     elif require_numeric and "numeric" not in (x_type, y_type):
   1631         err = "Neither the `x` nor `y` variable appears to be numeric."
-> 1632         raise TypeError(err)
   1633
   1634     else:

TypeError: Neither the `x` nor `y` variable appears to be numeric.
```

Proceso para obtener datos de Transfer Market:

- **Intento 1:** Buscar una Base de Datos Externa
  - Descripción: Intentamos encontrar una base de datos ya existente con datos de la vida real, pero no tuvimos éxito.
- **Intento 2**: Scraping de Transfer Market
  - Descripción: Decidimos hacer scraping de Transfer Market, ya que es una fuente confiable en el fútbol.
  - Desafío: Necesitábamos los IDs únicos de los jugadores asignados por Transfer Market para acceder a sus páginas individuales. No encontramos una manera automatizada de obtener estos IDs, por lo que consideramos "hardcodearlos" manualmente, pero esto no era viable para todos los jugadores debido a su gran número (aproximadamente 10 millones).
- **Intento 3**: Usar una API Externa
  - Descripción: Encontramos una API en GitHub que ya realizaba scraping de Transfer Market.
  - Desafío: No pudimos implementar la API y hacer las llamadas desde Google Cloud debido a nuestra inexperiencia en esta plataforma.
- **Solución Final**: Creación Propia de Script de Scraping
  - Descripción: Regresamos a la idea de crear nuestro propio script de scraping y limitamos el análisis a 71 jugadores con potencial igual o mayor a 89.
  - Resultados: Generamos gráficas del historial de valor de mercado de los 10 jugadores con mayor potencial, mostrando su evolución desde sus inicios en el fútbol hasta la actualidad.